

MICROPROCESSORS

TEXT BOOK:

Advanced Microprocessors and Peripherals - A.K. Ray and K.M. Bhurchandi, TMH, 3rd Edition, 2012, ISBN 978-1-25-900613-5.

REFERENCE BOOKS:

1. Microprocessor and Interfacing- Douglas V Hall, SSSP Rao, 3rd edition TMH, 2012.
2. Microcomputer systems-The 8086 / 8088 Family – Y.C. Liu and A. Gibson, 2nd edition, PHI -2003.

Unit 1

- **8086 Processor**
- Historical background, 8086 CPU Architecture.
Addressing modes, Machine language
instruction formats.
- **Instruction Set of 8086:**
- Data transfer and arithmetic instructions.,
Illustration of these instructions with example
programs

Unit 1

- **8086 Processor**
- Historical background, 8086 CPU Architecture.
Addressing modes, Machine language instruction formats.
- **Instruction Set of 8086:**
- Data transfer and arithmetic instructions.,
Illustration of these instructions with example programs

Historical Background

- The Mechanical Age
- The Electrical Age
- The Microprocessor Age
- The Intel Age – The Modern Microprocessor
 - 1970s
 - 1980s
 - 1990s
 - 2000s

The Mechanical Age

Dream: calculating with a machine

- The idea of computing system existed long before modern electrical and electronics devices were developed.
- The idea of calculating with a machine dates to 500 BC when the Babylonians invented the abacus, the first mechanical calculator, Abacus-string of beads used to perform calculations.

The Electrical Age

- First electronic (digital) computer- ENIAC-(Electronic numerical integrator calculator)
- Designed and developed at the U. of Pennsylvania, 1946
- contained over 18,000 vacuum tubes, weighed more than 30 tons, and required 1500 square feet of floor space and over 500 miles of wires
- It was programmed by setting up to 6000 switches and connecting cables between the various units of the computer.
- first modern (general-purpose, programmable) electronic computer performing about 100,000 operations per second
- programmed by rewiring its circuits

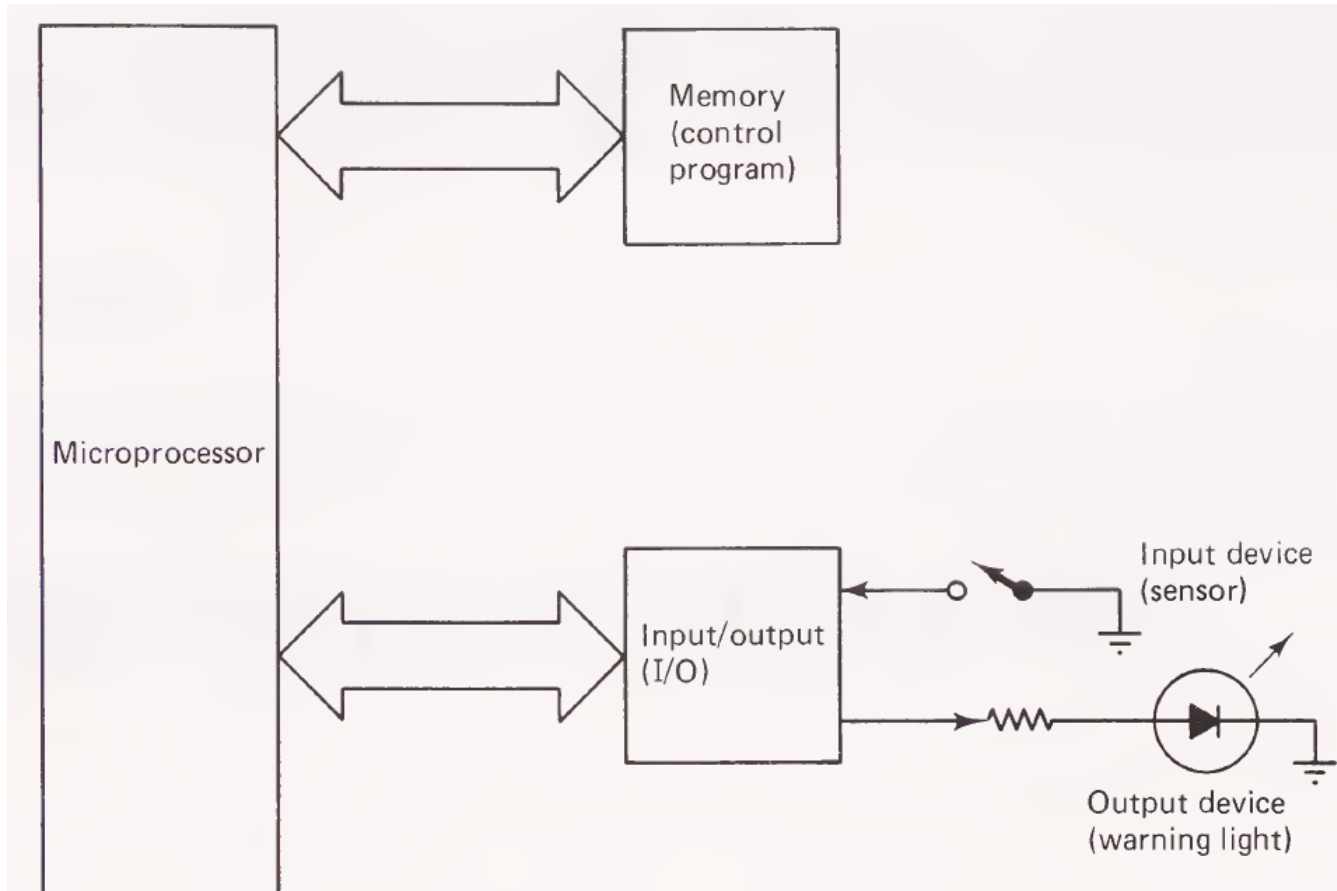
The Microprocessor Age

- Before the invention of microprocessors, most digital design problems were solved using the same **set of tools**.
- **Combinational circuits** were derived from Boolean logic equations, truth tables, and Karnaugh maps.
- **Sequential designs** utilized flip-flop excitation tables and state diagrams.
- Designers then chose “off the shelf” logic components —typically **7400 family integrated circuits**—to implement these equations in hardware.
- If the design specifications changed, the process was repeated and the hardware was modified or rebuilt.

The Microprocessor Age contd..

- Release of a new logic component called the **microprocessor** in the early 1970s changed all of this.
- Rather than implement a rigid set of logic equations, the microprocessor fetches from a memory unit, “instructions” about the problem to be solved.
- These instructions are called the control program.
- A microcomputer system is built around the microprocessor.

The Microprocessor Age contd..

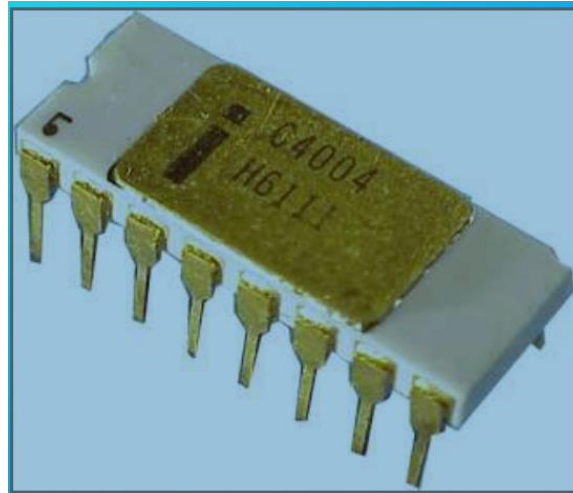


Typical microcomputer system, built around microprocessor

Source: *The 8086/8088 Family: Design, Programming and Interfacing* by John Uffenbeck

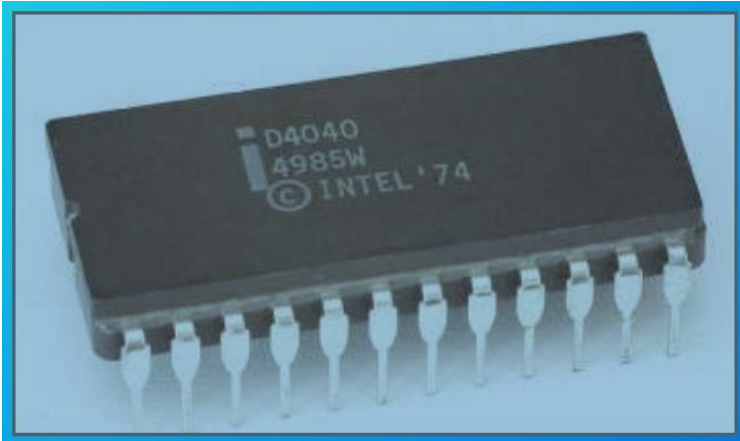
Intel 4004- 4 bit

- World's First microprocessor released in 1971
- Could execute 45 instructions
- 4096 location addressable (4 Kbytes) (2^{12})
- Weighed much less than one ounce $\cong$ 28gms.
- Initial clock speed: 108KHz
- Transistors: 2,300



Intel 4040- 4 bit

- Later Intel released 4040, updated version of 4004, which operated at higher speed
- Initial clock speed: 800 kHz
- Transistors-3500
- Lacked improvement in the word width and memory size. (4 bit, 4 Kbytes of memory)



Intel 8008- 8 bit



- Introduced in 1972
- Could execute 16kbytes of memory and 48 instructions.
- Initial clock speed-2 MHz
- Transistors-4500
- **Limitations:**
 - small memory size, slow speed, limited instruction set
 - To overcome the limitations, Intel released 8080, 1st modern 8bit microprocessor-1973
 - About 6 months later Motorola released MC6800, other companies began to release their own versions of 8 bit microprocessor.

Intel 8080- 8 bit

- Introduced in 1973
- first modern 8-bit microprocessor
- Could address more memory and additional instructions
- 10 times faster than 8008
- Ex: addition that took 20μsec on 8008 took 2μsec on 8080
- Addressed 4 times more memory than 8008 (64 Kbytes)

8086 Register Usage

General Registers

- AX (AH,AL): Accumulator, Math operations (General Purpose)
- BX (BH,BL): Address/Pointer
- CX(CH,CL): Counting & Looping
- DX(DH,DL): Data, Most Significant bytes of a 16-bit MUL/DIV

Index Registers

- SP: Stack pointer
- BP: Base pointer, Stack operations: parameters of a subroutine
- SI: Source Index (arrays & strings)
- DI: Destination Index (arrays & strings)

Segment Registers

- CS: Code Segment: Used with Instruction Pointer (IP) to fetch instructions
- DS: Default (Data) Segment: Variables
- SS: Stack Segment: Subroutines & local variables in procedures
- ES: Extra Segment: String operations

General Data Registers (1)

- AX, BX, CX, DX (4 registers)
- 8 bit / 16 bit

AX : Accumulator

- Most commonly used (important)
- AX : Word multiply, word divide, word I/O
- AL: Byte multiply, Byte divide, Byte I/O translate, decimal arithmetic, AH: same with AL

CX : Counter Register

- Default counter (String , Loop Operations)
- CL : Positions in shift & rotate instructions

BX : Base Register

- Offset storage for forming physical addresses in addressing modes

DX : Data Register

- Implicit operand / destination
- Word multiply, word divide, to hold port addresses in I/O operation



Pointers and Index Registers (2)

- 5 registers: IP, BP, SP , SI, DI

Instruction Pointer (IP)

- 16 bit ; Not direct control of programmer
- Identifies the location of the **next word of instruction code** to be fetched from current segment of memory
- The actual address is obtained by combining its content with CS register
- In a normal operation, 8086 , fetch codes from Code segment of memory and store in instruction queue and execute one after one. Each time a code is fetched, it update IP
- This is followed by decoding and execution

Stack Pointer (SP)

- Used in stack operation (?)
- Always points to 'top' of the stack
- Usage combined with SS

Pointers and Index Registers-II(2)

Base Pointer (BP)

- Base Pointer (BP) is a 16-bit register pointing to data in stack segment.
- BP register is usually used for based, based indexed or register indirect addressing.

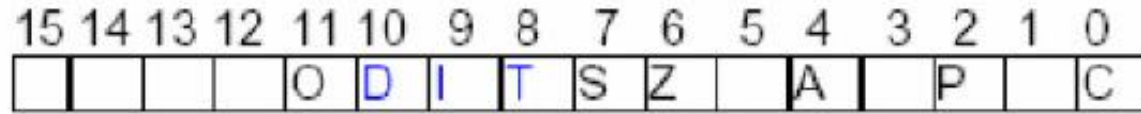
Index Pointers (SI, DI)

- Source Index (SI) is a 16-bit register.
- SI is used for indexed, based indexed and register indirect addressing, as well as a source data address in string manipulation instructions.
- Destination Index (DI) is a 16-bit register.
- DI is used for indexed, based indexed and register indirect addressing, as well as a destination data address in string manipulation instructions.

The segment registers(CS,DS,SS,ES)

- **Code segment register(CS)**-used to define the code segment where the program instructions currently being executed are located.
- **Data segment register(DS)**-used to define the data segment where the program data are located
- **Stack Segment register(SS)**-used to define the segment used for stack
- **Extra segment register(ES)**-is an additional data segment that is used by some of the string instructions to hold destination data

Flag Register (4)



- Indicates the results of computations / condition of microprocessor
- A 16-bit flag register in the EU contains nine active flags

1) conditional flags-6

- They are set or reset by the EU on the basis of the results of last arithmetic operation.
- Carry Flag (CF)- set by carry out of MSB.
- Parity Flag (PF)- set if result has even parity.
- Auxiliary carry Flag (AF)- for BCD
- Zero Flag (ZF)- set if result = 0
- Sign Flag (SF) = MSB of result
- Overflow Flag (OF)- overflow flag

Conditional flags-Examples

If the following addition is carried out,

$$\begin{array}{r} 0010\ 0011\ 0100\ 0101 \\ + 0011\ 0010\ 0001\ 1001 \\ \hline 0101\ 0101\ 0101\ 1110 \end{array}$$

the FLAGS register's condition flags are set as follows:

S (sign) = 0, Z (zero) = 0, P (parity) = 0, C (carry) = 0,

A (aux carry) = 0, O (overflow) = 0

If instead the following were performed,

$$\begin{array}{r} 0101\ 0100\ 0011\ 1001 \\ + 0100\ 0101\ 0110\ 1010 \\ \hline 1001\ 1001\ 1010\ 0011 \end{array}$$

the FLAGS register's condition flags would be set as follows:

S (sign) = 1, Z (zero) = 0, P (parity) = 1, C (carry) = 0,

A (aux carry) = 1, O (overflow) = 1

Flag register contd..

2) control flags: The control flags enable or disable certain operations of the microprocessor

- The three remaining flags in the flags register are used to control certain operations of processor.
- TF- single step trap flag
- IF- interrupt enable flag
- DF- string direction flag

Control flags

- Direction flag- In certain string manipulation instructions, D determines whether the string is processed from the lowest address (D=0) or the highest address (D=1)
- D=0: Auto- increment, D=1: Auto-decrement
- I (Interrupt flag)-determines whether a maskable interrupt is recognized by the microprocessor.
- If I=1, a maskable interrupt is possible, otherwise the interrupt is ignored.
- T (Trap flag)-IF T=1, a trap (eg. For single stepping through a program) is executed after every instruction.

Definition of Microprocessor

- “The integrated circuit (IC) which contains all the function of the CPU (Central Processing Unit) of a computer is known as Microprocessor.”
- Heart of the microprocessor based computer system

Intel 8085

- Introduced in 1977
- Updated version of 8080
- Could execute software at higher speed
- Ex: Addition that took 2 μsec on 8080 required only 1.3 μsec on 8085

Main Advantages

- Internal clock generator
- Internal system controller
- High clock frequency

Intel 8086

- Introduced in 1978
- Transistors: 29,000
- Initial clock speed: 5MHz
- Could execute instructions in as little as 400nsec (2.5 MIPS)
- Addressed 1Mbytes of memory , 16 times more memory than 8085
- **Higher execution speed** and larger memory size allowed 8086 to replace smaller mini computers in many applications

Terms

- KIP – Kilo of Instructions per Second
- MIP – Millions of Instructions per Second
- Bit - a binary digit with a value of 1 or 0
- Nibble – 4-bit-wide binary number
- Byte – 8-bit-wide binary number
- word (16 bits)
- Doubleword (32 bits)
- Quadword (64 bits)
- Octalword (128 bits)
- K – 1024, 1M – 1024K, 1G – 1024M

Intel Microprocessors Evolution

- 4004
- 8008
- 8080
- 8085
- 8086
- 80286
- 80386
- 80486
- Pentium
- Pentium II
- Pentium III
- Pentium 4
- Intel Core duo
- Quad core...

80186

- Basically 8086 with on chip priority controller, programmable timer, DMA controller & address decoding circuit
- Mostly used in industrial control applications

80286

- Enhancement of 8086
- Almost identical to 8086/88 except it addresses 16Mbytes of memory instead of 1Mbytes
- 4MIPS(250ns/8MHZ)-executes some instructions in 250nsec

80386(1986)

- First practical 32bit processor with 32bit data bus and 32bit memory address
- Can address 4GB main memory
- 12-33MHZ
- Memory management unit is added-hardware cktry for memory mgmt-reduces software overhead.

- 80486(1989)**

- 32 bit processor, 32 bit data bus and 32 bit address bus

- 4GB main memory

- 20-50MHz, later at 66 and 100MHz

- Incorporated an 80386 like microprocessor and 80387 like floating point coprocessor and an 8Kbyte cache on one package.

- About half of the instructions executed in 1 clock instead of 2 on 80386

- For a clock freq of 50MHz, half of the instructions executed in 25nsec.(50MIPS)

- Later versions of 486, executed instructions at even higher speeds.

- Variations: SX, DX2, DX4

- DX2-doubled clocked version

- 66MHZ clock cycle time with memory transfer at 33 MHZ

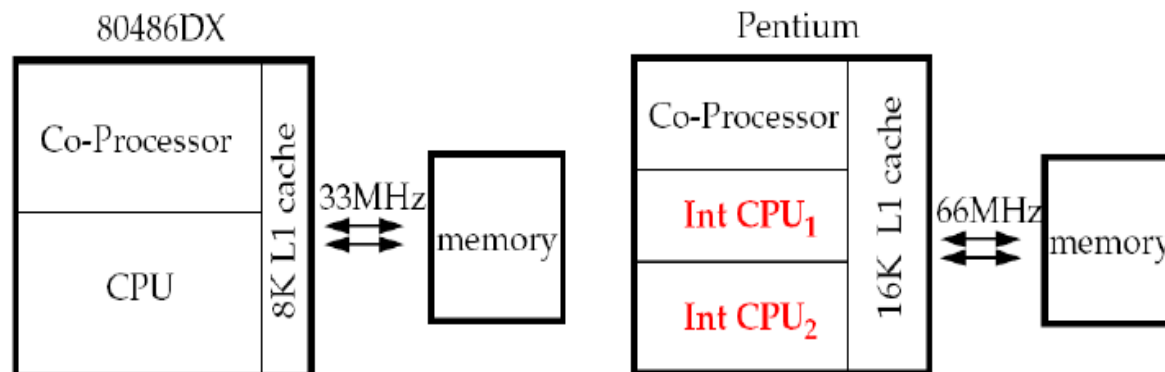
- 80486(1989)**

- 32 bit processor, 32 bit data bus and 32 bit address bus
- 4GB main memory
- 20-50MHZ, later at 66 and 100MHz
- Incorporated an 80386 like microprocessor and 80387 like floating point coprocessor and an 8Kbyte cache on one package.
- About half of the instructions executed in 1 clock instead of 2 on 386
- For a clock freq of 50Mhz, half of the instructions executed in 25nsec.(50MIPS)
- Later versions of 486, executed instructions at even higher speeds.
- Variations: SX, DX2, DX4
- DX2-doubled clocked version
- 66MHZ clock cycle time with memory transfer at 33 MHZ

80x86 Evolution

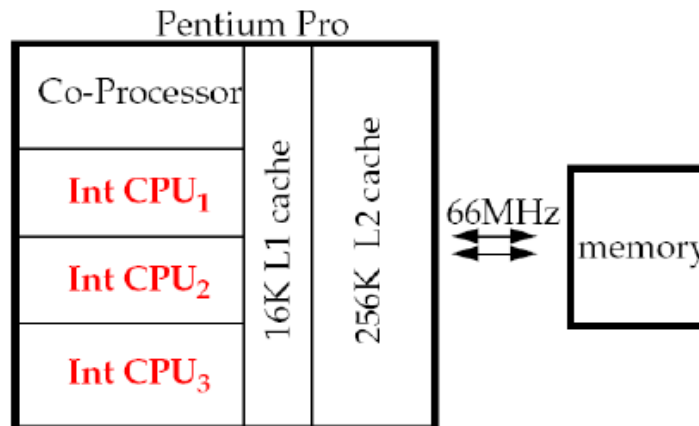
Pentium (1993)

- 32 bit microprocessor, 64 bit data bus and 32 bit address bus
- 4GB main memory
- 60,66,90MHz
- 16KB L1 cache
- Memory transfers at 66 MHz instead of 33MHz
- Dual Integer processors



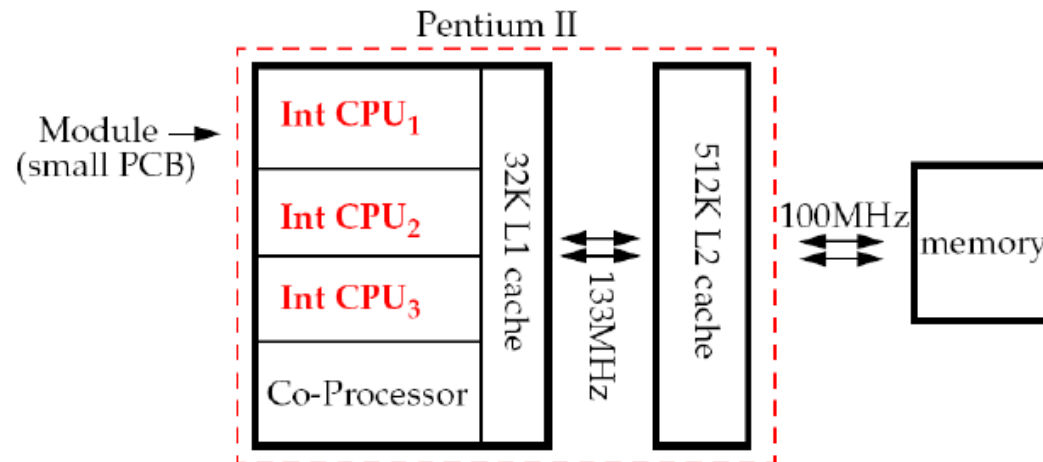
Pentium Pro (1995)

- 32-bit microprocessor, 64-bit data bus and 36 bit address bus
- 64GB main memory
- Starts at 150MHz
- 16KB L1 cache (split instruction/data:8KB each)
- 256 KB L2 cache
- Memory transfers at 66 MHz
- 3 integer processors



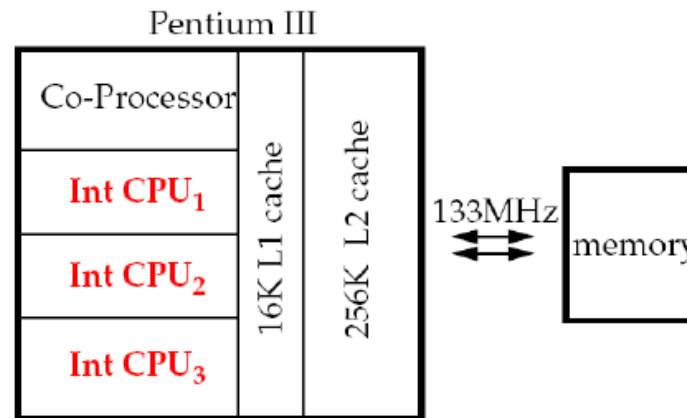
Pentium II(1997)

- 32-bit microprocessor, 64-bit data bus and 36-bit address bus
- 64GB main memory
- Starts at 266 MHz
- 32KB split instruction/data: L1 caches (16 KB each)
- Module integrated 512 KB L2 cache (133MHz)
- Memory transfers at 66 MHz to 100 MHz



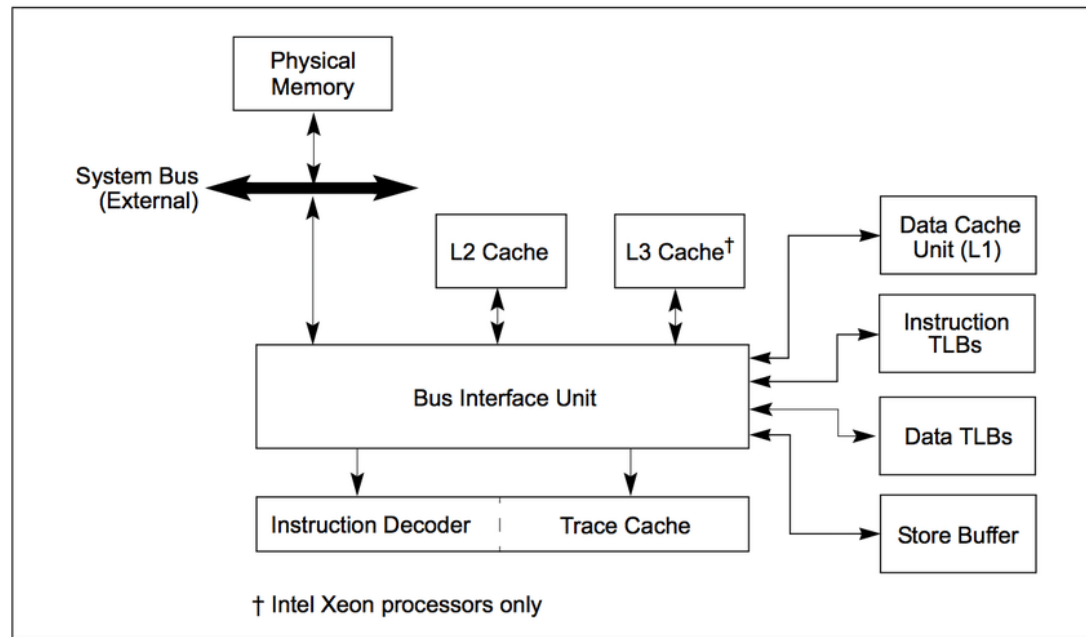
Pentium III (1999)

- 32-bit microprocessor, 64-bit data bus and 36-bit address bus
- 64GB main memory
- Up to 800MHz (faster clock than PII)
- 32KB split instruction/data: L1 caches (16 KB each)
- On-chip 256 KB L2 cache
- Memory transfers 100 MHz to 133 MHz
- Dual Independent Bus (Simultaneous L2 and system memory access)



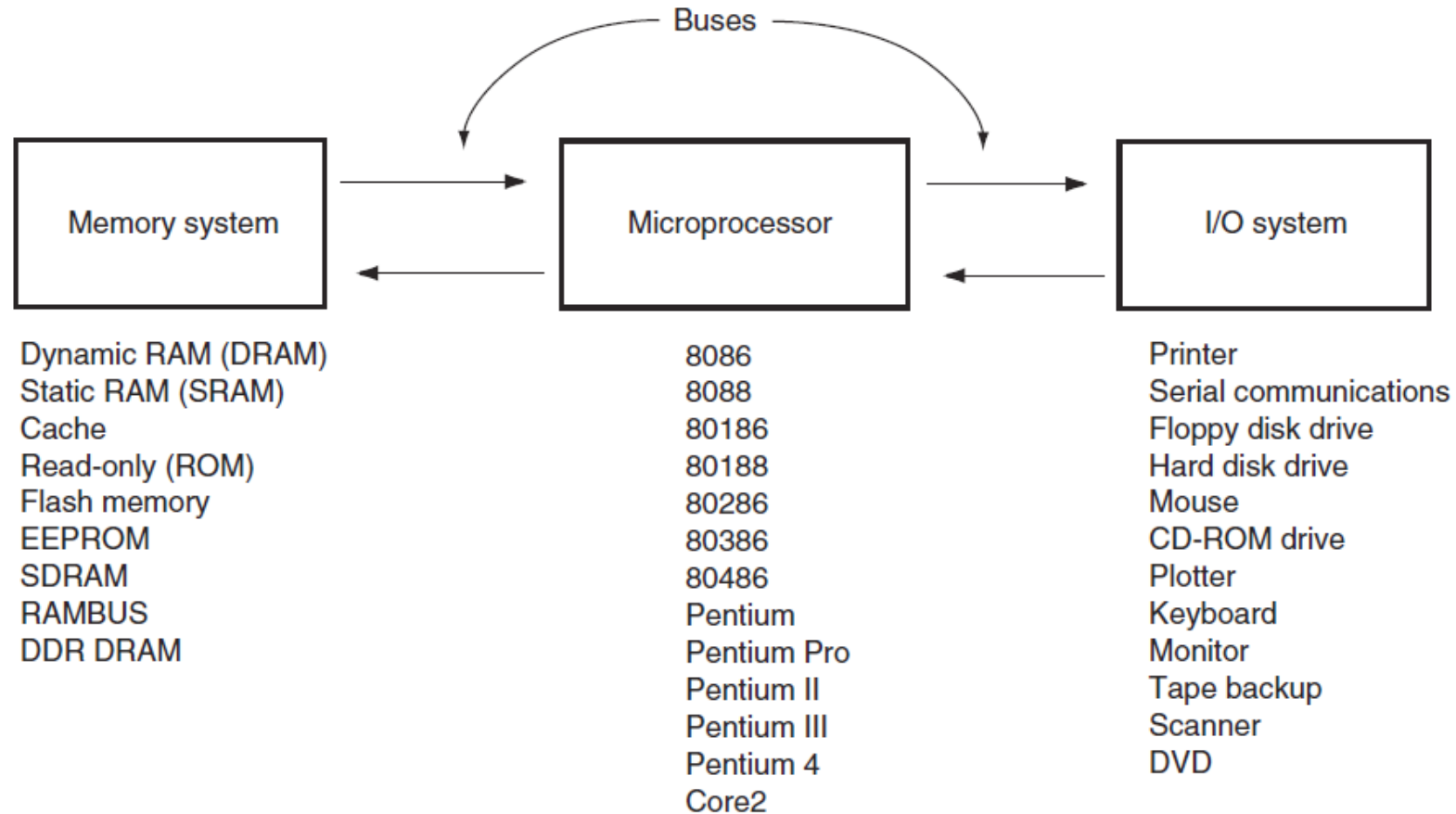
Pentium IV (2002)

- 1.4-1.9GHz and the latest at 2.2 GHz
- 512KB L2 cache
- 400MHz system bus
- Specialized for streaming video, game and DVD apps

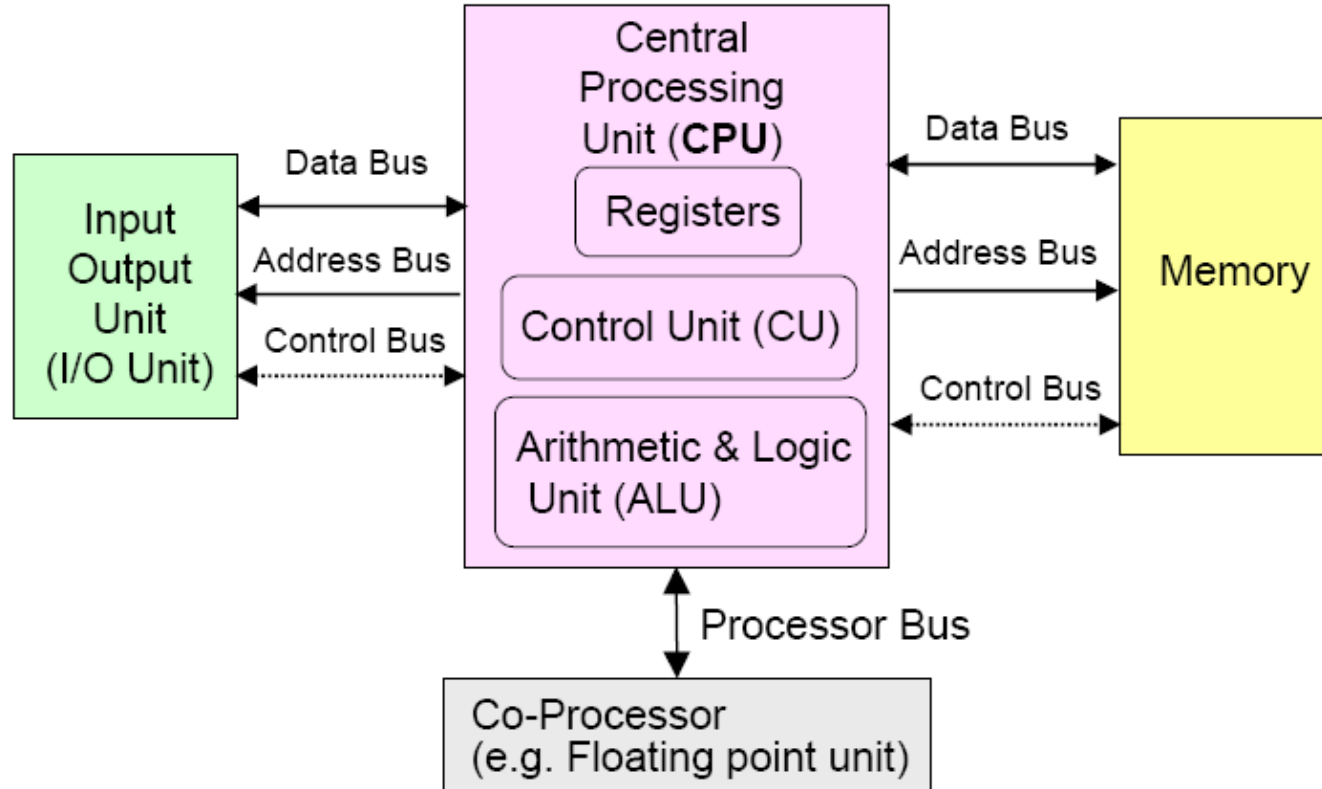


TLB-Translation lookaside buffer –used to reduce the time taken to access the user memory location

The Microprocessor-based Personal Computer System



The Microprocessor-based Personal Computer System



The Microprocessor-based Personal Computer System contd..

- Control Unit: Generates all the control signals within the CPU
- It initializes the registers on power-up, generates the signal to fetch instructions for the ALU
- Registers: Usually stores data and addresses associated with the instruction being carried out
- ALU-performs arithmetic and logical operations

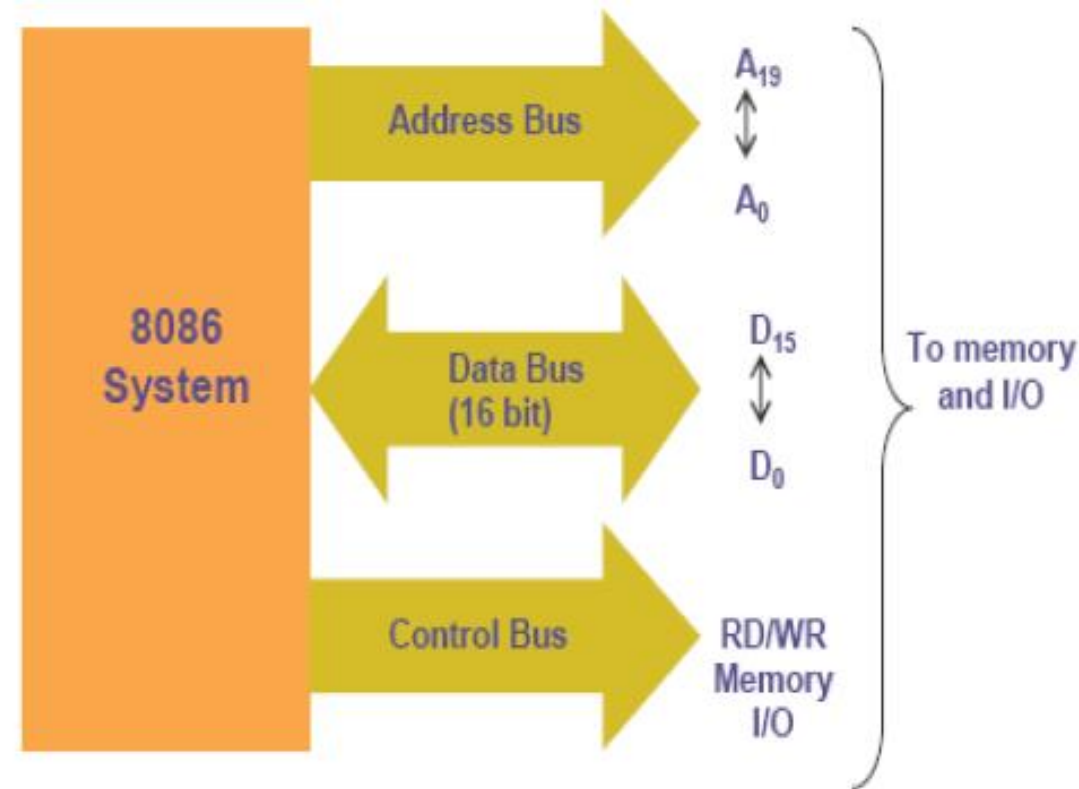
Buses

A common group of wires that interconnect components in a computer system

Address Bus provides a memory address to the system memory and I/O address to the system I/O devices

Data Bus transfers data between the microprocessor and the memory and I/O attached to the system

Control Bus provides control signals that cause the memory or I/O to perform a read or write operation



Microprocessor

- Controlling element in a computer
- Controls memory and I/O devices through a series of connections called buses.
- Can be defined in many ways
 - programmable logic device that can be used to control processes and to turn on/off devices.
 - viewed as a data processing unit or computing unit of a computer
- Performs 3 tasks
 1. Data transfer between itself and memory or I/O devices
 2. Simple arithmetic and logical operations
 3. Program flow via simple decisions.

- The heart of microprocessor-based computer system
 - million instructions per seconds
 - variable data widths
 - 8086-80286: directly manipulate 8- and 16-bit numbers
 - 80386 or higher: 32-bit numbers
 - 80486 or higher: floating-point arithmetic
- program flow via simple decisions
- determine numerical conditions
- branch on the conditions
- control the I/O and memory system

Bus Architecture

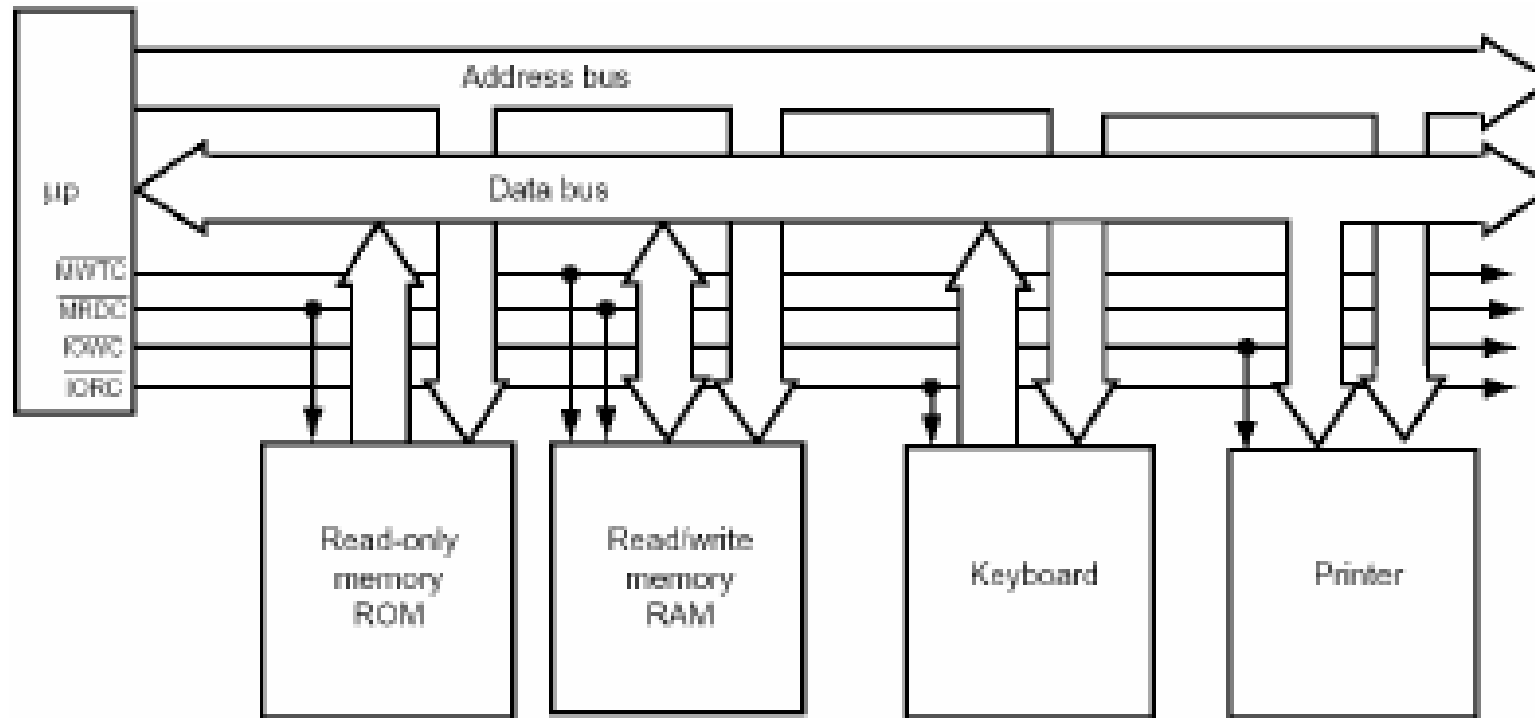
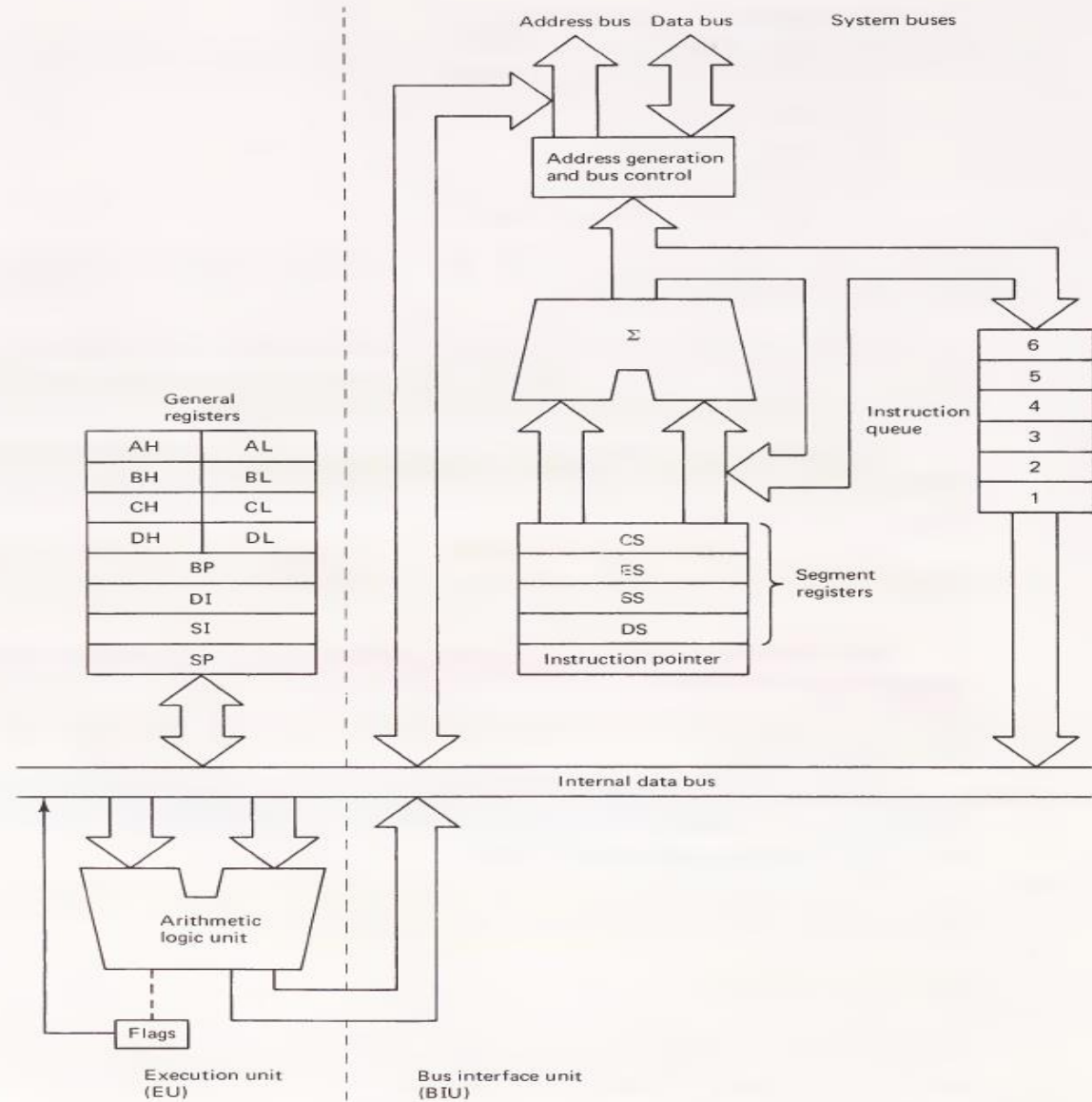


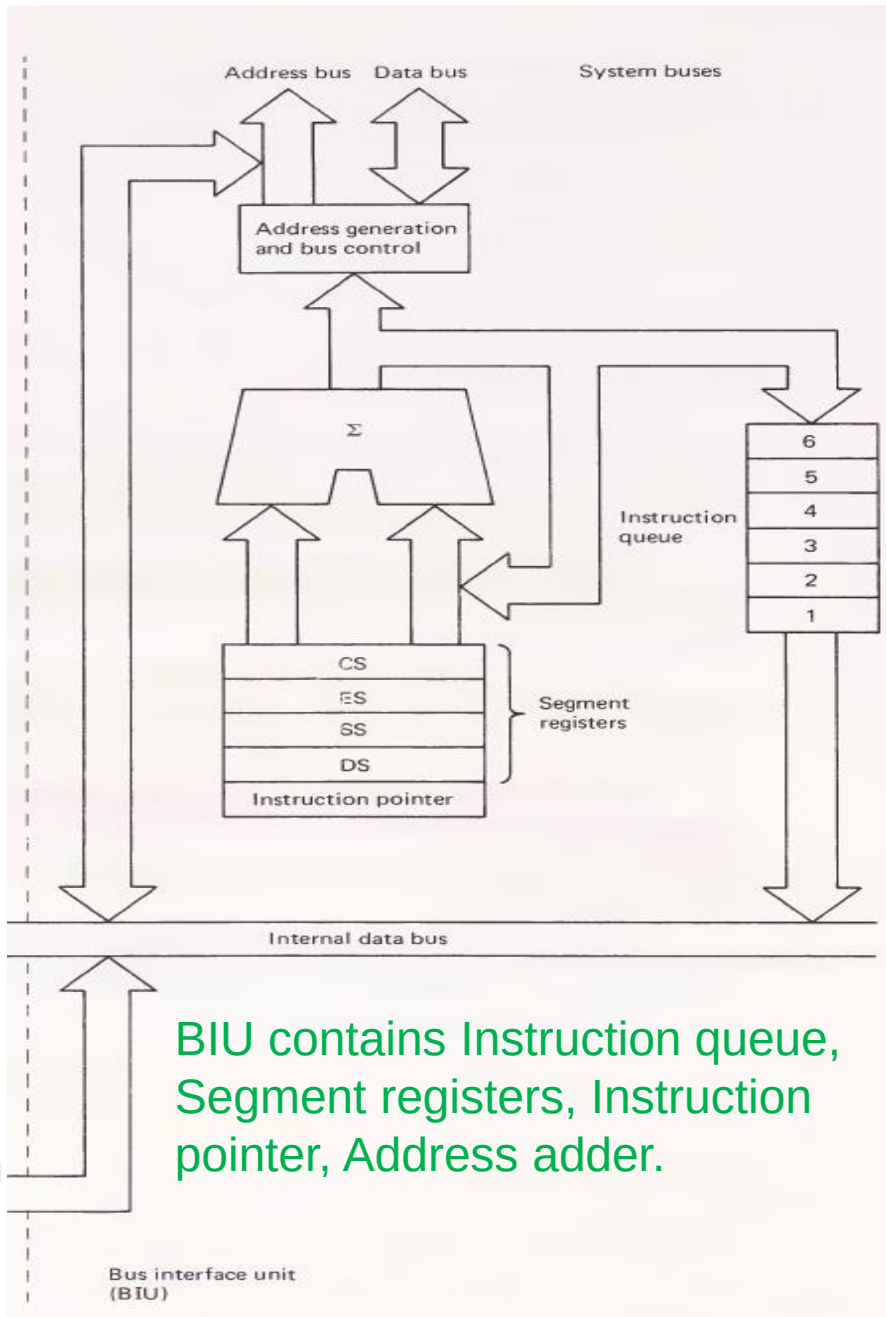
FIGURE 1-11 The block diagram of a computer system showing the address, data, and control bus structure.

- Four basic control bus connections (over-bar = active-low)
- MRDC (Memory Read Control)
 - MWTC (Memory Write Control)
 - IORC (I/O Read Control)
 - IOWC (I/O Write Control)

Architecture of 8086 microprocessor

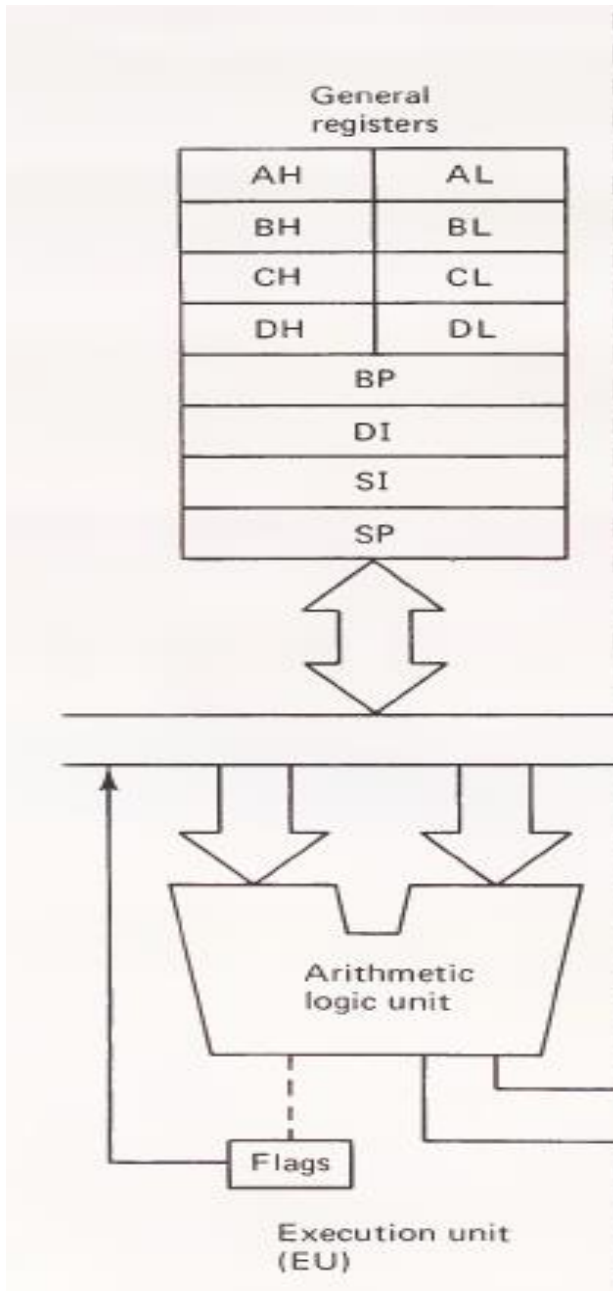


BIU



- provides hardware functions, including generation of the memory and I/O addresses for the transfer of data between the outside world—outside the CPU, and the EU.
- **Performs all bus operations** such as instruction fetching, reading and writing operands for memory and calculating the addresses of the memory operands.
- The instruction bytes are transferred to the instruction queue.

EU



- receives program instruction codes and data from the BIU, executes these instructions, and stores the results in the general registers.
- By passing the data back to the BIU, data can also be stored in a memory location or written to an output device.
- EU has no connection to the system buses.
- It receives and outputs all its data through the BIU.

EU contains Control circuitry, Instruction decoder, ALU, Pointer and Index register, Flag register.

Data Transfer Instructions

- PUSH Operand: decrements SP by 2 and Copies a word from some source to the location in the SS where the SP then points
- It pushes the operand into top of stack.
- E.g.: PUSH BX BH→[SP-1], BL→[SP-2]
- SP=SP-2
- PUSH [BX]
- [Bx+1]→[SP-1] MSB
- [BX]→[SP-2] LSB
- POP Des:
- It pops the operand from top of stack to Des.
- Des can be a general purpose register, segment register (except CS) or memory location.
- E.g.: POP AX

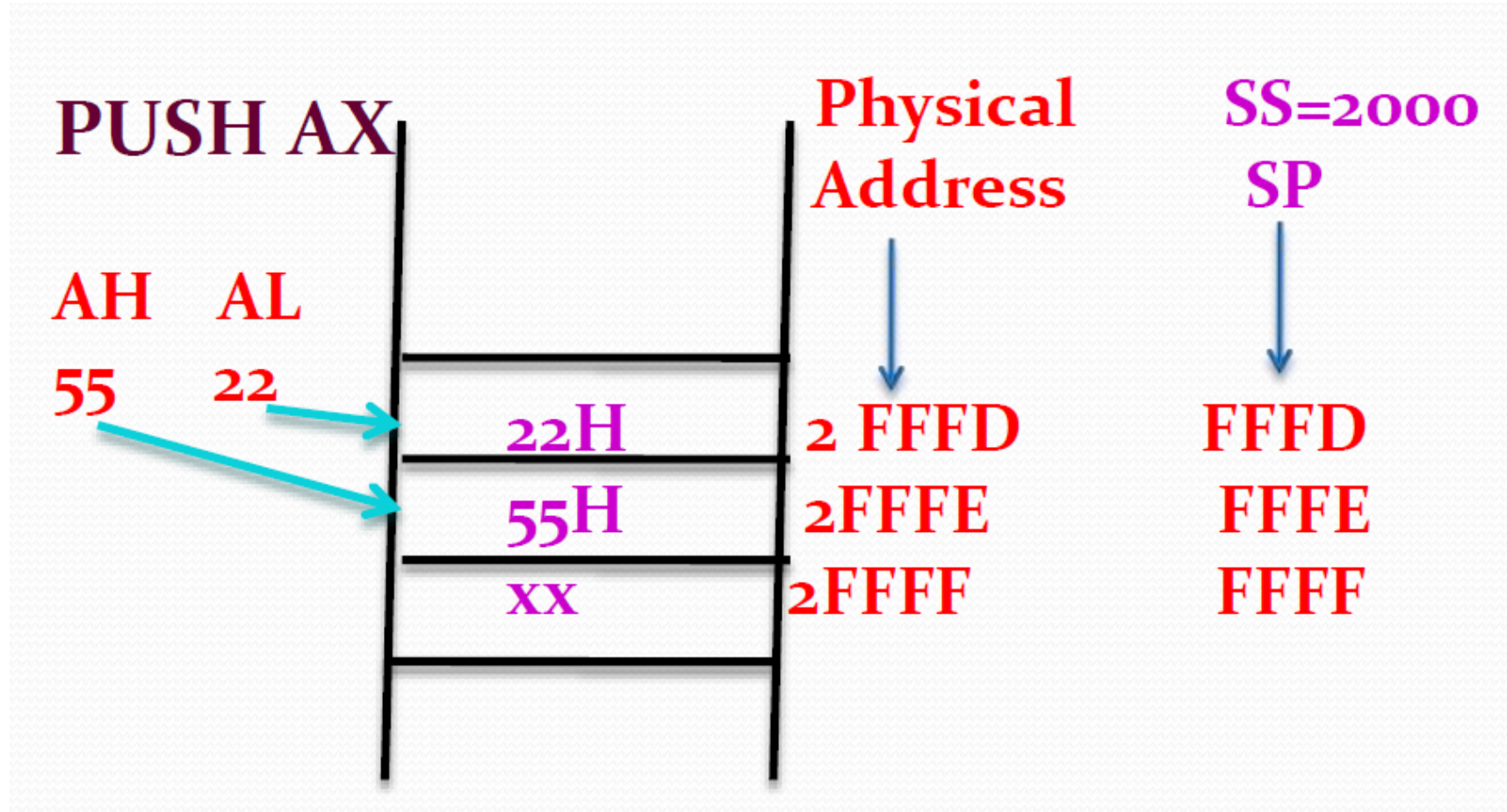
Table 1-1 Data Transfer Group [except SAHF, no other instruction affects the flag]

Mnemonic	Description	Clock Cycles	No. of bytes	Example / Remarks
MOV dst, src	Move ;			
	Register to Register	2	2	MOV AL, BH
	Register to Memory	9 + EA	2-4	MOV [5200], CX
	Memory to Register	8 + EA	2-4	MOV DL, [BX]
	Immediate data to Register	4	2-3	MOV AX, 34AB
	Immediate data to Memory	10 + EA	3-6	MOV [SI], 03A4
	Register to SS, DS, ES register	2	2	MOV DS, BX
	Memory to SS, DS, ES register	8 + EA	2-4	
	Segment register to register	2	2	
	Segment register to Memory	9 + EA	2-4	
PUSH src	Push word onto stack ;			
	Register	11	1	PUSH BX
	Segment register (except CS)	10	1	PUSH DS
	Memory	16 + EA	2-4	PUSH LOC [BX]
POP dst	Pop word off stack ;			
	Register (16-bit)	8	1	POP CX
	Segment register (except CS)	8	1	POP ES
	Memory	17 + EA	2-4	POP NUM [SI]
XCHG dst, src	Exchange; (seg regs not used)			
	Register with Accumulator	3	1	XCHG AX, CX
	Register with Register	4	2	XCHG BL, DH
	Register with Memory	17 + EA	2-4	XCHG [BX], CL
IN Acc, port	Read from Input port; Read data from port into AL or AX	10	2	IN AL, 0F8
	Fixed port	8	1	IN AX, 45
	Variable port			IN AL, DX
OUT port, Acc	Write to Output port; Put data from AL or AX to output port	10	2	OUT 80, AL
	Fixed port	8	1	OUT 0A5, AX
	Variable port			OUT DX, AX
LEA Reg, src	Load Effective Address; Finds the offset of the variable or mem location named as source, and puts this in the given 16-bit Register.	2 + EA	2-4	LEA BX, TABLE LEA SI, STRING LEA CX, [BX]
LDS/LES Reg, Mem addr	Load Pointer using DS or ES; Loads new values into specified register and DS or ES reg from 4 successive memory locations.	16 + EA	2-4	Useful for string operations LDS BX, [5683] LES DI, [BX] LDS SI, STRING
LAHF	Load AH register with lower byte of Flag register	4	4	Saving Flag register in register
SAHF	Store AH into lower byte of Flag register	1	1	(Change flag settings manually)
PUSHF	Push Flag register on stack	10	1	Save Flag status on stack
POPF	Pop flags off stack	8	1	Retrieve Flag status
XLAT/ XLATB	Translate; Replaces a byte in AL with a byte pointed to by BX in a lookup table in memory. Can be used for code conversion	11	1	LEA BX, TABLE XLATB

PUSH Instruction

- Pushes the contents of specified register/memory location on to the stack.
- Decrements the stack pointer by 2 and copies a word from a specified source to the location in the stack segment to which the stack pointer points.
- This instruction does not affect any flag.
- Ex. PUSH BX Decrement SP by 2, copy BX to stack.
- PUSH DS Decrement SP by 2, copy DS to stack.

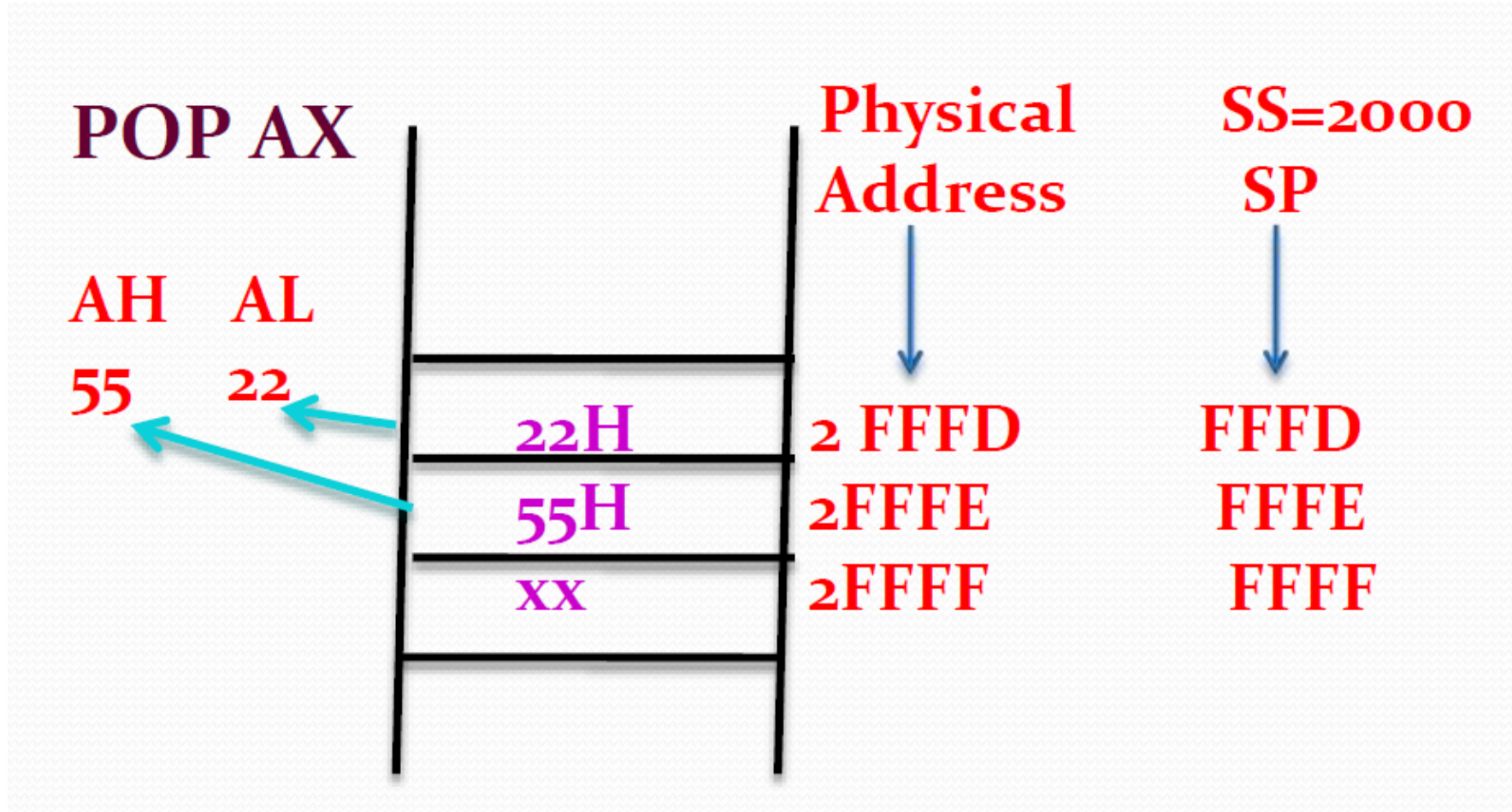
PUSH instruction Example



POP instruction

- The POP instruction copies a word from the stack location pointed to by the stack pointer to a destination specified in the instruction.
- The destination can be a general-purpose register, a segment register or a memory location.
- The data in the stack is not changed. After the word is copied to the specified destination, the stack pointer is automatically incremented by 2 to point to the next word on the stack.
- The POP instruction does not affect any flag.
- Ex-POP DX .
- Copy a word from top of stack to DX; increment SP by 2

PUSH instruction Example



Use of XLAT instruction

- EXAMPLE: A TABLE OF ASCII CODES FOR HEXADECIMAL DIGITS
- DATA SEGMENT
- HA_TABLE DB '0123456789ABCDEF' ;This generates the table
- H_DIGIT DB 7 ;Assume we want the ASCII code for a 7
- ASC_DIGIT DB ? ;Store ASCII code here
- DATA ENDS
- CODE SEGMENT
- ASSUME DS:DATA,CS:CODE
- MOV BX,OFFSET HA_TABLE ;Offset of table
- MOV AL,H_DIGIT ;Position of table entry
- XLAT ;Now AL contains ASCII code
- MOV ASC_DIGIT,AL ;Store it
- RET
- CODE ENDS
- END MAIN

AL <--- [AL+BX]

Execution unit

- Executes instructions from the instruction system byte queue.
- Both units operate asynchronously to give the 8086 an overlapping instruction fetch and execution mechanism which is called as Pipelining.

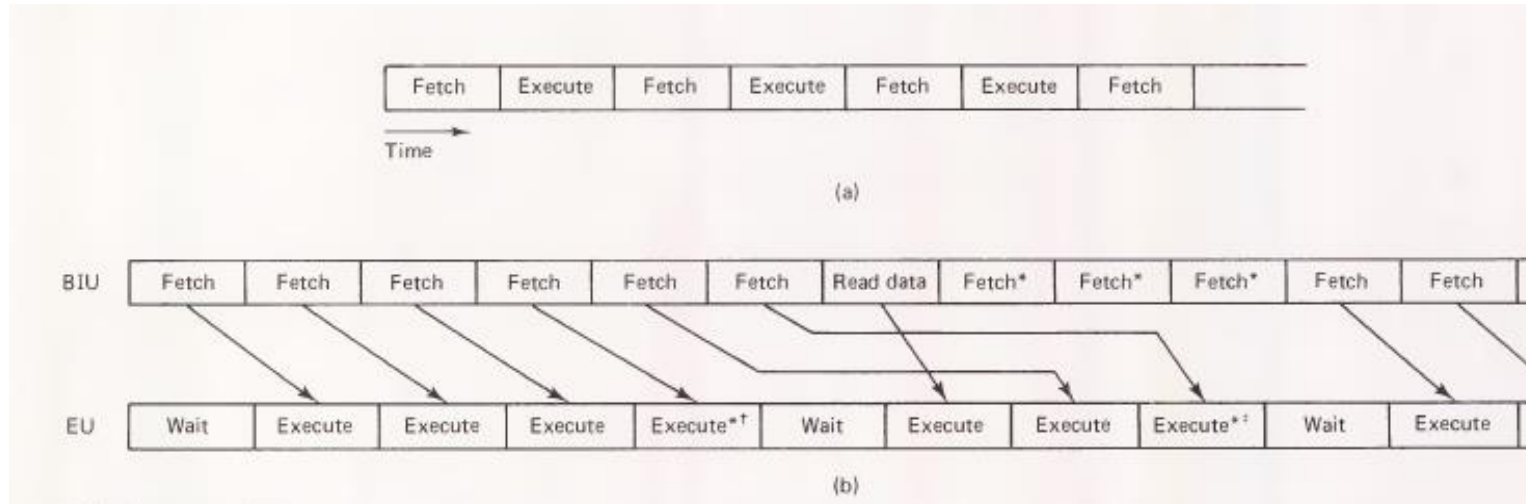
Bus Interface Unit (BIU)

- It provides a full 16 bit bidirectional data bus and 20 bit address bus.
- The bus interface unit is responsible for performing all external bus operations.
- ***Specifically it has the following functions:***
- Instruction fetch, Instruction queuing, Operand fetch and storage, Address relocation and Bus control.

Bus Interface Unit (BIU) contd..

- The BIU uses a mechanism known as an instruction stream queue to implement a ***pipeline architecture***.
- This queue permits prefetch of up to six bytes of instruction code.
- When ever the queue of the BIU is not full, it has room for at least two more bytes and at the same time the EU is not requesting it to read or write operands from memory, the BIU is free to look ahead in the program by prefetching the next sequential instruction.

Pipelined Architecture



*These bytes are discarded

*[†] This instruction requires a request for data **not** in the queue

*[‡] Jump instruction occurs

Figure 2.2 (a) The conventional (nonpipelined) microprocessor follows a sequential fetch and execute cycle. (b) The 8086/88's pipelined architecture allows the EU to execute instructions without the delays associated with instruction fetching.

8086 Register Usage

General Registers

- AX (AH,AL): Accumulator, Math operations (General Purpose)
- BX (BH,BL): Address/Pointer
- CX(CH,CL): Counting & Looping
- DX(DH,DL): Data, Most Significant bytes of a 16-bit MUL/DIV

Index Registers

- SP: Stack pointer
- BP: Base pointer, Stack operations: parameters of a subroutine
- SI: Source Index (arrays & strings)
- DI: Destination Index (arrays & strings)

Segment Registers

- CS: Code Segment: Used with Instruction Pointer (IP) to fetch instructions
- DS: Default (Data) Segment: Variables
- SS: Stack Segment: Subroutines & local variables in procedures
- ES: Extra Segment: String operations

MUL – MUL Source

- This instruction multiplies an *unsigned* byte in some *source* with an *unsigned* byte in AL register or an unsigned word in some *source* with an unsigned word in AX register.
- The source can be a register or a memory location.
- When a byte is multiplied by the content of AL, the result (product) is put in AX.
- When a word is multiplied by the content of AX, the result is put in DX and AX registers.
- If the most significant byte of a 16-bit result or the most significant word of a 32-bit result is 0, CF and OF will both be 0's. *AF, PF, SF and ZF are undefined after a MUL instruction.*

Examples

- MUL BH; Multiply AL with BH; result in AX
 - MUL CX ;Multiply AX with CX; result high word in DX, low word in AX
 - MUL BYTE PTR [BX]; Multiply AL with byte in DS pointed to by [BX]
 - MUL FACTOR [BX]; Multiply AL with byte at effective address FACTOR [BX], if it is declared as type byte with DB. Multiply AX with word at effective address FACTOR [BX], if it is declared as type word with DW.
-
- MOV AX, MCAND_16; Load 16-bit multiplicand into AX
 - MOV CL, MPLIER_8 ;Load 8-bit multiplier into CL
 - MOV CH, 00H ;Set upper byte of CX to all 0's
 - MUL CX ; multiply AX and CX; 32-bit result in DX and AX

DIV – DIV Source

- This instruction is used to divide an *unsigned* word by a byte or to divide an *unsigned* double word (32 bits) by a word.
- When a word is divided by a byte, the word must be in the AX register. The divisor can be in a register or a memory location. After the division, AL will contain the 8-bit quotient, and AH will contain the 8-bit remainder.
- DIV BL ; Divide word in AX by byte in BL; Quotient in AL, remainder in AH
- DIV [BX]; Divide word in AX by byte in memory pointed to by BX
- DIV SCALE [BX] ; (byte at effective address SCALE [BX]) if SCALE [BX] is of type byte;

DIV – DIV Source

- When a double word is divided by a word, the most significant word of the double word must be in DX, and the least significant word of the double word must be in AX.
- DIV CX ;Divide double word in DX and AX by word in CX; Quotient in AX, and remainder in DX
- After the division, AX will contain the 16-bit quotient and DX will contain the 16-bit remainder.
- If an attempt is made to divide by 0 or if the quotient is too large to fit in the destination (greater than FFH / FFFFH), the 8086 will generate a type 0 interrupt.
- All flags are undefined after a DIV instruction.

DIV – DIV Source

- If you want to divide a byte by a byte, you must first put the dividend byte in AL and fill AH with all 0's.
- Likewise, if you want to divide a word by another word, then put the dividend word in AX and fill DX with all 0's.
- `DIV SCALE [BX]` ; (DX and AX) / (word at effective address `SCALE[BX]` if `SCALE[BX]` is of type word

INC – INC Destination

- The INC instruction adds 1 to a specified register or to a memory location.
- AF, OF, PF, SF, and ZF are updated, but CF is not affected.
- This means that if an 8-bit destination containing FFH or a 16-bit destination containing FFFFH is incremented, the result will be all 0's with no carry.
- INC BL ; Add 1 to content of BL register
- INC CX ;Add 1 to content of CX register
- INC BYTE PTR [BX] ;Increment byte in data segment at offset contained in BX.
- INC WORD PTR [BX] ;Increment the word at offset of [BX] and [BX + 1] in the data segment.

DEC – DEC Destination

- This instruction subtracts 1 from the destination word or byte.
- The destination can be a register or a memory location.
- AF, OF, SF, PF, and ZF are updated, but CF is not affected.
- If an 8-bit destination containing 00H or a 16-bit destination containing 0000H is decremented, the result will be FFH or FFFFH with no carry (borrow).

Subtract 1 from byte or word named COUNT in DS.

- DEC CL Subtract 1 from content of CL register
- DEC BP Subtract 1 from content of BP register
- DEC BYTE PTR [BX] Subtract 1 from byte at offset [BX] in DS.
- DEC WORD PTR [BP] Subtract 1 from a word at offset [BP] in SS.

DAA (DECIMAL ADJUST AFTER BCD ADDITION)

- This instruction is used to make sure the result of adding two packed BCD numbers is adjusted to be a BCD number.
- The result of the addition must be in AL for DAA to work correctly.
- If the lower nibble in AL after an addition is greater than 9 or AF was set by the addition, then the DAA instruction will add 6 to the lower nibble in AL.
- If the result in the upper nibble of AL is now greater than 9 or if the carry flag was set by the addition or correction, then the DAA instruction will add 60H to AL.
- Let AL = 59 BCD, and BL = 35 BCD
- ADD AL, BL AL = 8EH; lower nibble > 9, add 06H to AL
- DAA AL = 94 BCD, CF = 0
- Let AL = 88 BCD, and BL = 49 BCD
- ADD AL, BL AL = D1H; AF = 1, add 06H to AL
- DAA AL = D7H; upper nibble > 9, add 60H to AL
- AL = 37 BCD, CF = 1
- The DAA instruction updates AF, CF, SF, PF, and ZF; but OF is undefined

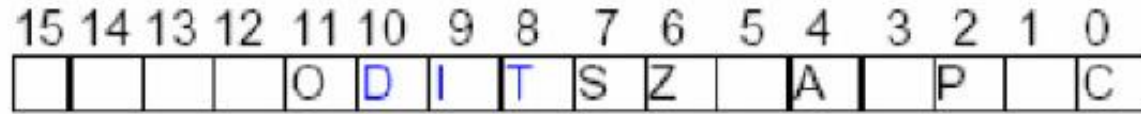
DAS (DECIMAL ADJUST AFTER BCD SUBTRACTION)

- This instruction is used after subtracting one packed BCD number from another packed BCD number, to make sure the result is correct packed BCD.
- The result of the subtraction must be in AL for DAS to work correctly.
- If the lower nibble in AL after a subtraction is greater than 9 or the AF was set by the subtraction, then the DAS instruction will subtract 6 from the lower nibble AL.
- If the result in the upper nibble is now greater than 9 or if the carry flag was set, the DAS instruction will subtract 60 from AL.
- Let AL = 86 BCD, and BH = 57 BCD
- SUB AL, BH ; AL = 2FH; lower nibble > 9, subtract 06H from AL
- AL = 29 BCD, CF = 0

Example 2

- Let AL = 49 BCD, and BH = 72 BCD
- SUB AL, BH AL = D7H; upper nibble > 9, subtract 60H from AL
- DAS AL = 77 BCD, CF = 1 (borrow is needed)
- The DAS instruction updates AF, CF, SF, PF, and ZF; but OF is undefined

Flag Register (4)



- Indicates the results of computations / condition of microprocessor
- A 16-bit flag register in the EU contains nine active flags

1) conditional flags-6

- They are set or reset by the EU on the basis of the results of last arithmetic operation.
- Carry Flag (CF)- set by carry out of MSB.
- Parity Flag (PF)- set if result has even parity.
- Auxiliary carry Flag (AF)- for BCD
- Zero Flag (ZF)- set if result = 0
- Sign Flag (SF) = MSB of result
- Overflow Flag (OF)- overflow flag

Conditional flags-Examples

If the following addition is carried out,

$$\begin{array}{r} 0010\ 0011\ 0100\ 0101 \\ + 0011\ 0010\ 0001\ 1001 \\ \hline 0101\ 0101\ 0101\ 1110 \end{array}$$

the FLAGS register's condition flags are set as follows:

S (sign) = 0, Z (zero) = 0, P (parity) = 0, C (carry) = 0,

A (aux carry) = 0, O (overflow) = 0

If instead the following were performed,

$$\begin{array}{r} 0101\ 0100\ 0011\ 1001 \\ + 0100\ 0101\ 0110\ 1010 \\ \hline 1001\ 1001\ 1010\ 0011 \end{array}$$

the FLAGS register's condition flags would be set as follows:

S (sign) = 1, Z (zero) = 0, P (parity) = 1, C (carry) = 0,

A (aux carry) = 1, O (overflow) = 1

Flag register contd..

2) control flags: The control flags enable or disable certain operations of the microprocessor

- The three remaining flags in the flags register are used to control certain operations of processor.
- TF- single step trap flag
- IF- interrupt enable flag
- DF- string direction flag

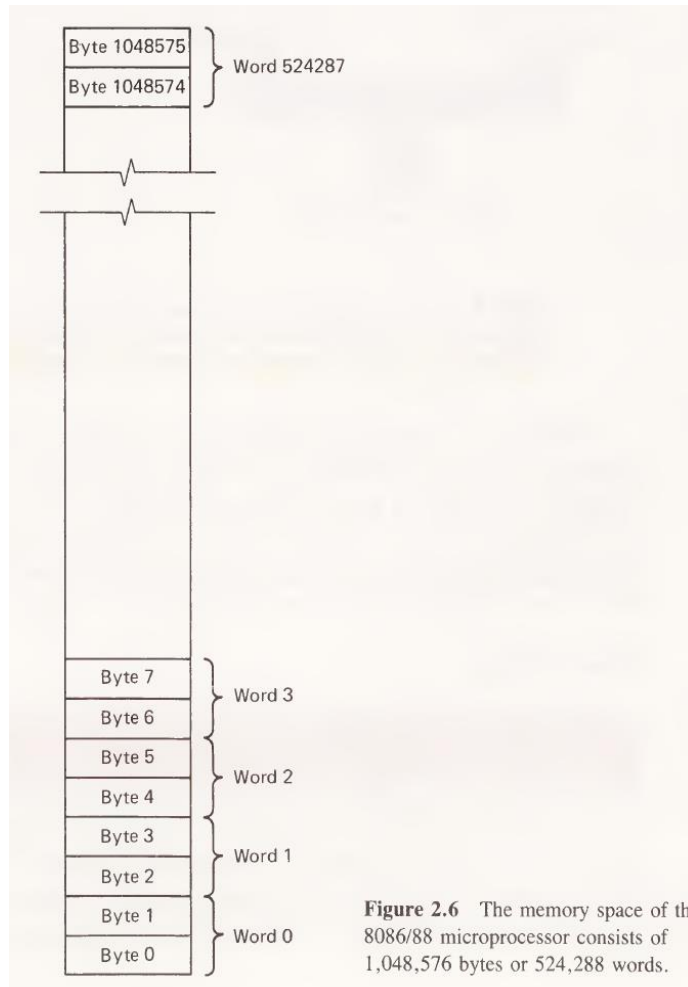
Control flags

- Direction flag- In certain string manipulation instructions, D determines whether the string is processed from the lowest address (D=0) or the highest address (D=1)
- D=0: Auto- increment, D=1: Auto-decrement
- I (Interrupt flag)-determines whether a maskable interrupt is recognized by the microprocessor.
- If I=1, a maskable interrupt is possible, otherwise the interrupt is ignored.
- T (Trap flag)-IF T=1, a trap (eg. For single stepping through a program) is executed after every instruction.

Segmented Memory

- Even though the 8086 is a 16-bit microprocessor, (it has a 16-bit data bus width), each memory location can store a byte and not word.
- First, it allows the processor to work on bytes as well as words.
- This is especially important with I/O devices such as printers, terminals, and modems, all of which are designed to transfer ASCII-encoded (7- or 8-bit) data.
- Second, many of the 8086's operation codes are single bytes.
- Other instructions may require anywhere from two to seven bytes. By being able to access individual bytes, these odd-lengthed instructions can be handled.

Segmented Memory contd..

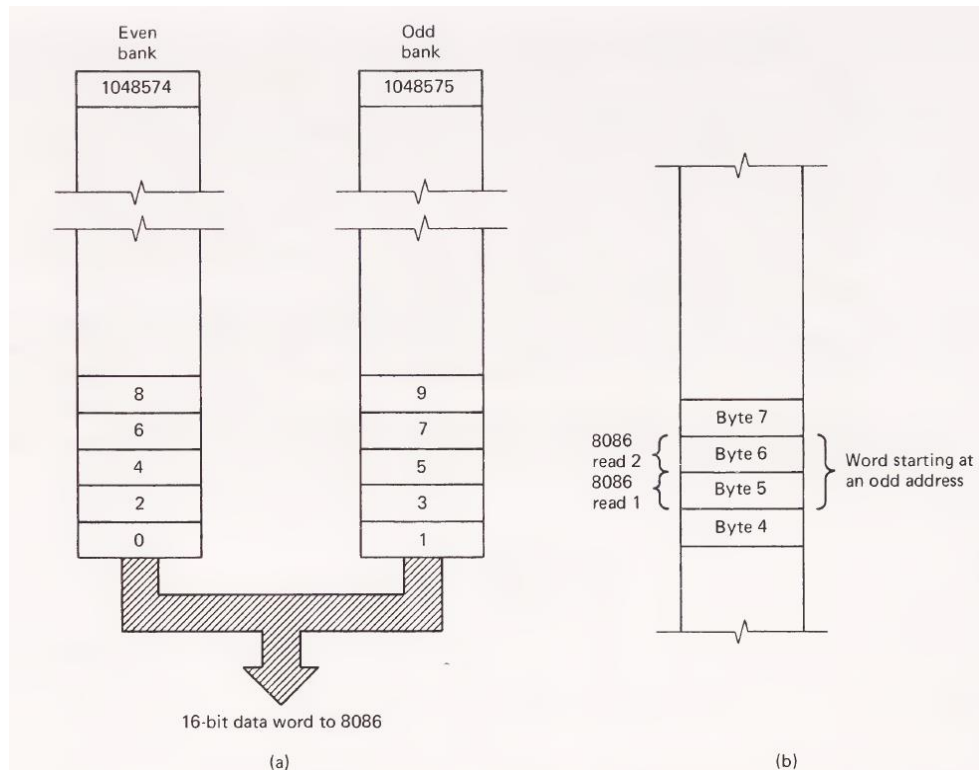


$$2^{20} = 1048576 \text{ bytes}$$

Figure 2.6 The memory space of the 8086/88 microprocessor consists of 1,048,576 bytes or 524,288 words.

Segmented Memory contd..

- 8086 reads 16 bits from memory by simultaneously reading an odd-addressed byte and an even-addressed byte.
- For this reason the 8086 organizes its memory into an even-addressed bank and an odd-addressed bank



Addressing modes

The way in which the operand is specified

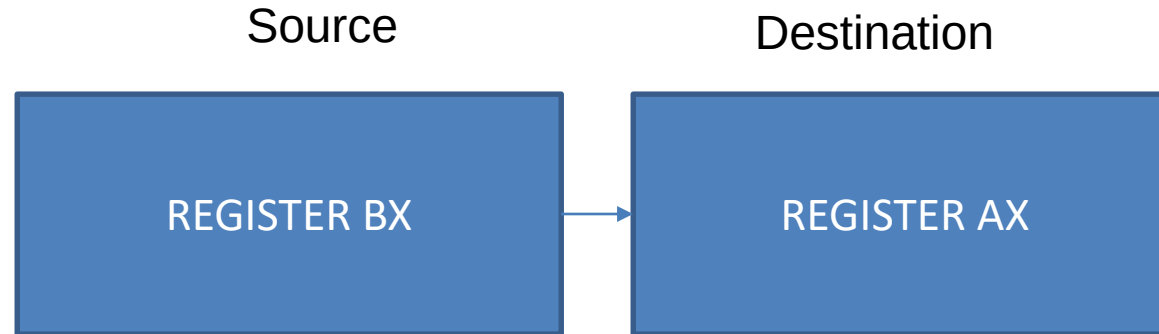
- Register addressing mode
- Immediate addressing mode
- Direct addressing mode
- Register indirect addressing mode
- Base +Index addressing mode
- Register relative addressing mode
- Base relative+Index addressing mode

Register addressing

- Transfers a copy of a byte/word from the source register to the destination register

Examples:

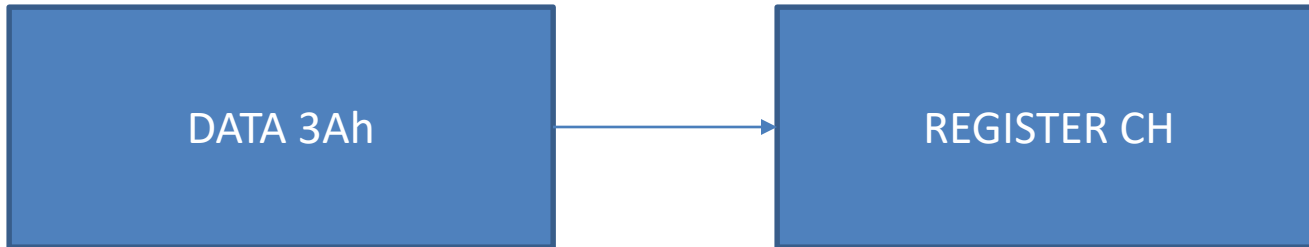
- MOV BX, AX
- MOV DX, CX
- MOV AL, AH
- MOV CL, AH
- MOV SI, DI
- MOV SP, BP



Immediate addressing

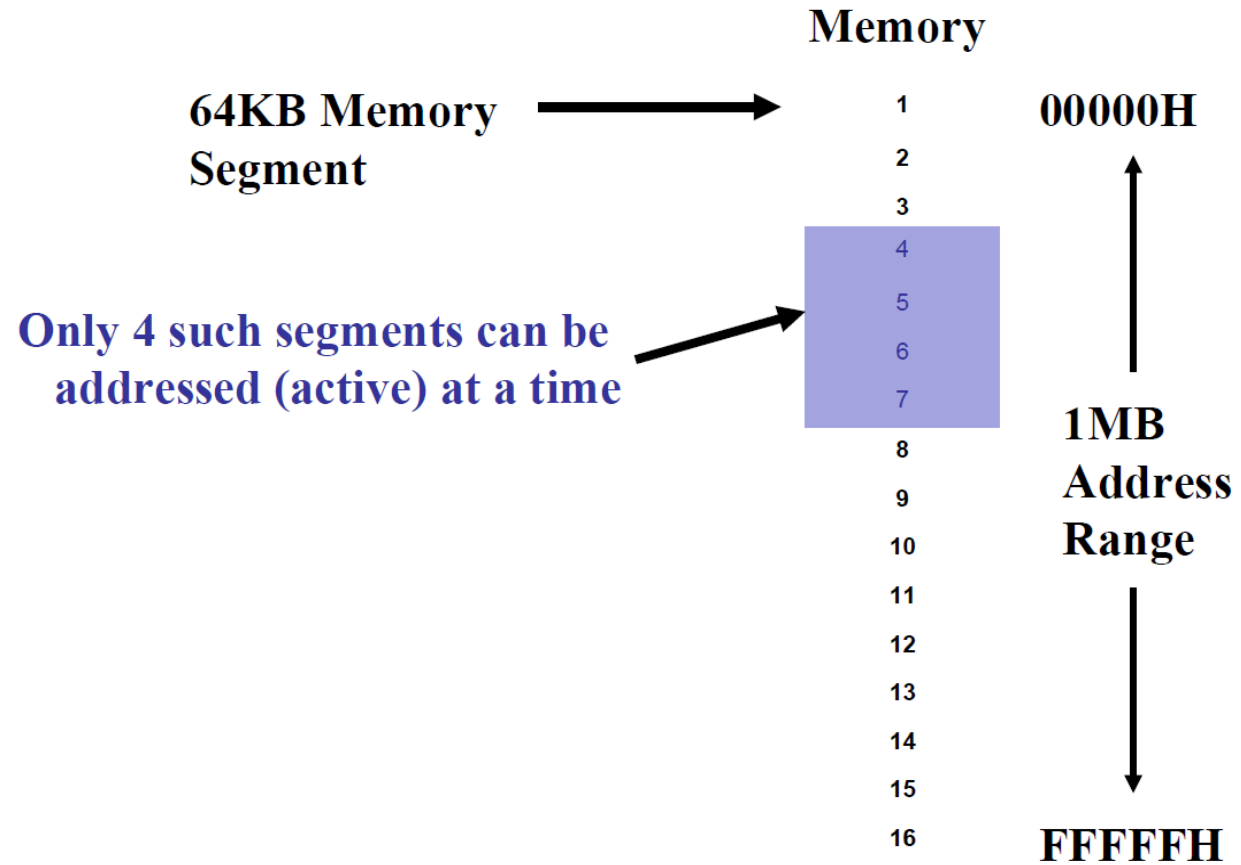
- Instruction itself specifies the data
- Data appears in the memory location immediately after the opcode
- Operates on word or byte
- Instruction format

Ex: MOV CH, 3Ah



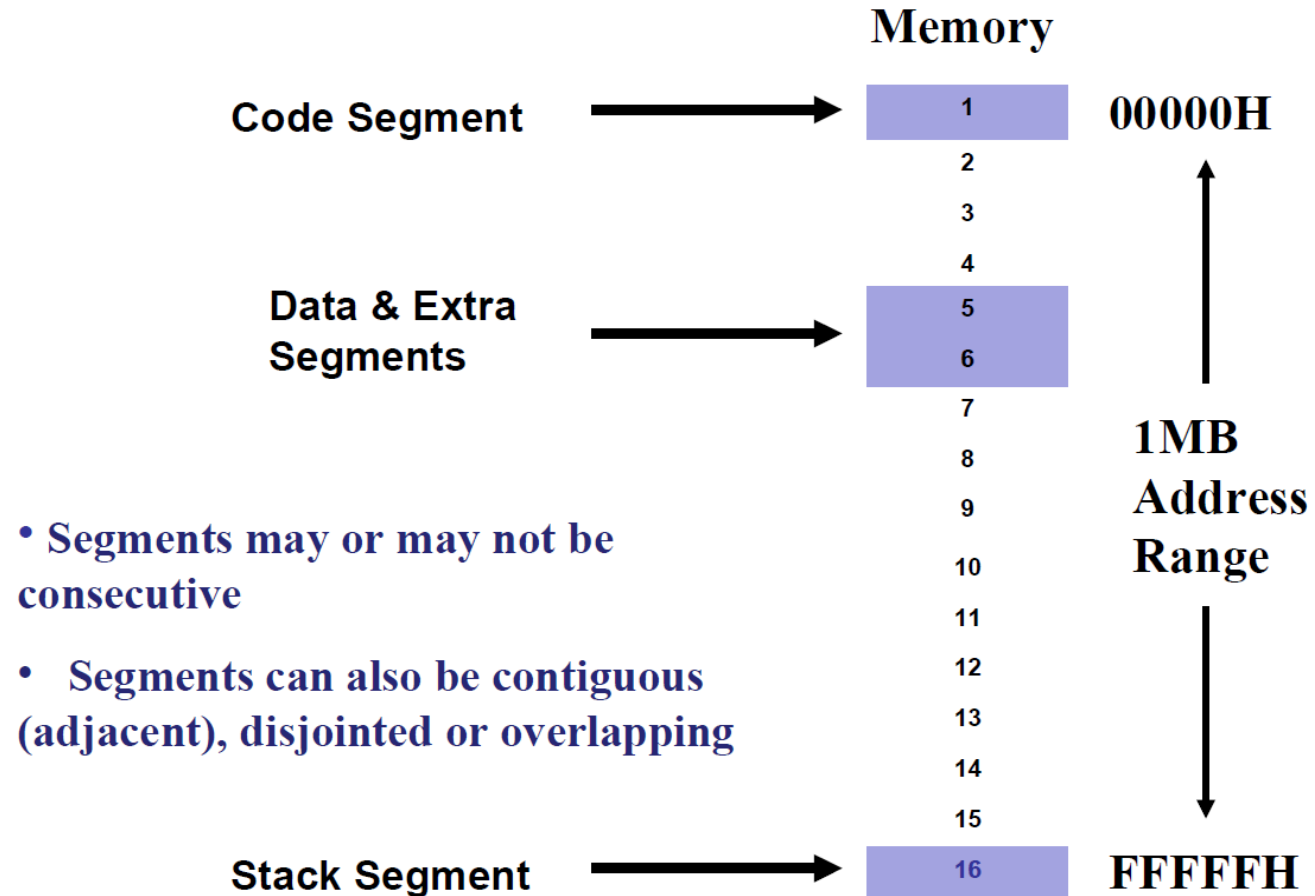
- MOV AX, 0888h
- MOV CH, 100
- MOV AL, 'A'

Memory Segmentation

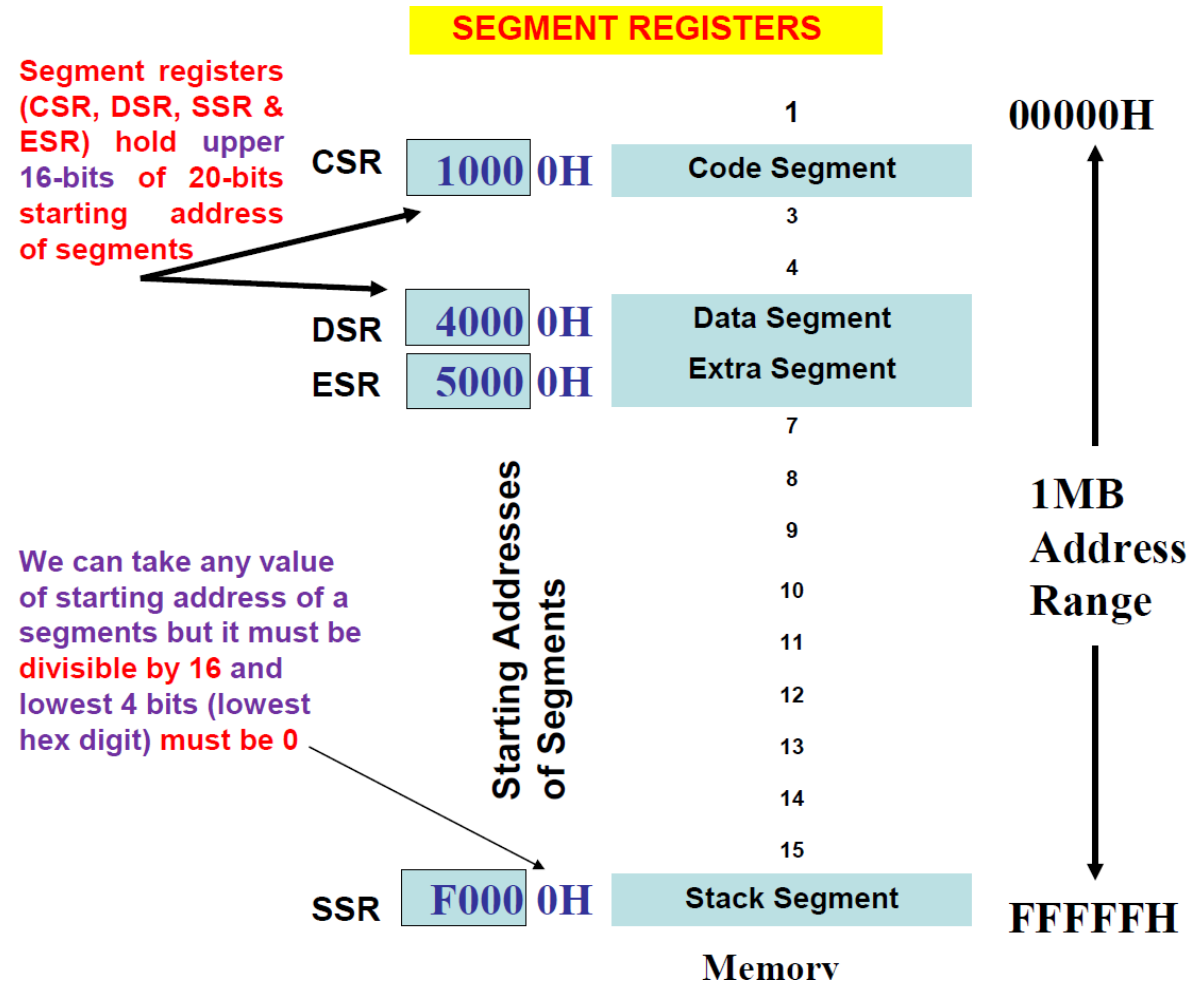


00000-1048575d

Memory Segmentation

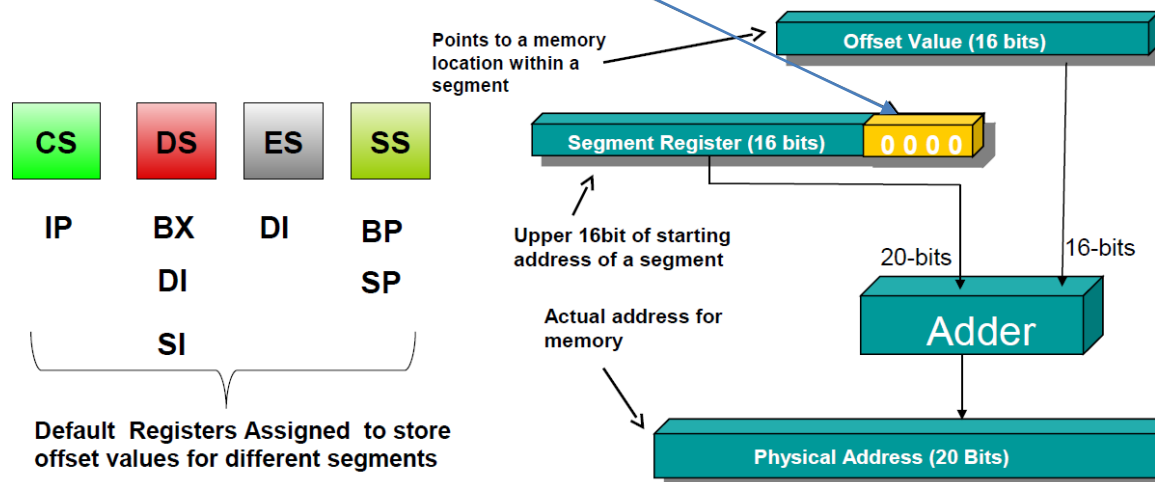


Memory Segmentation



Physical address generation in 8086

- The 20-bit physical address is generated by adding 16-bit contents of a segment register with a 16-bit offset value (also called Effective address) which is stored in corresponding default register (IP, BX, SI, DI, BP or SP)
- Different segments have different default register for offset
- Ex: IP is default register for code segment
- BIU always appends **4 zeros** automatically to the 16-bit address of the segment register (to make it 20 bit) because it knows that the starting address of the segment always ends with 4 zeros.



Physical Address Calculation

- ❑ **Offset** is derived from the combination of pointer registers, index registers the Instruction Pointer, and immediate values (called *displacement*)

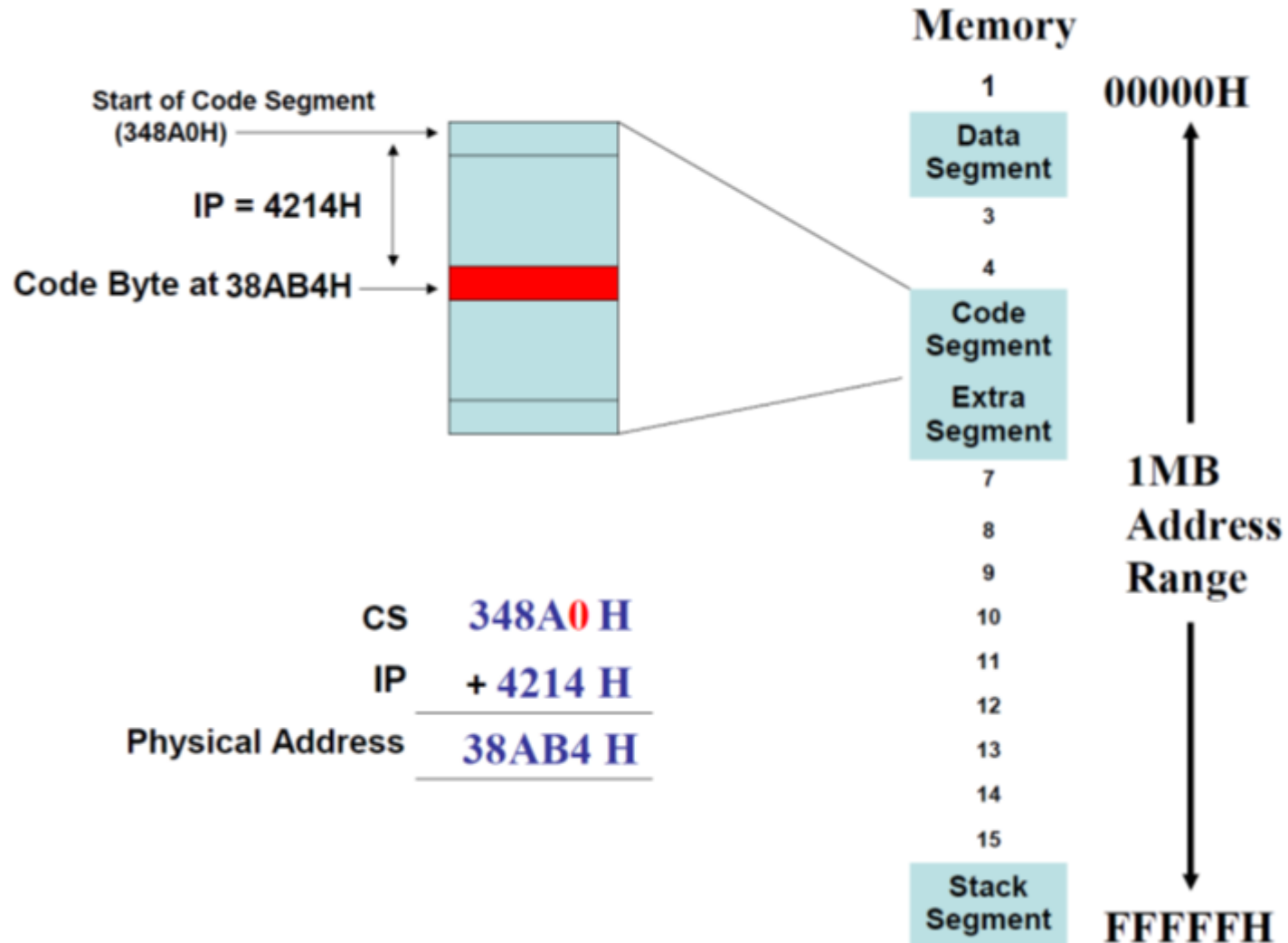
$$\begin{array}{r}
 \boxed{\text{Segment address}} \quad 0000 \\
 + \quad \boxed{\text{Offset}} \\
 \hline
 \boxed{\text{Memory address}}
 \end{array}$$

❑ Examples

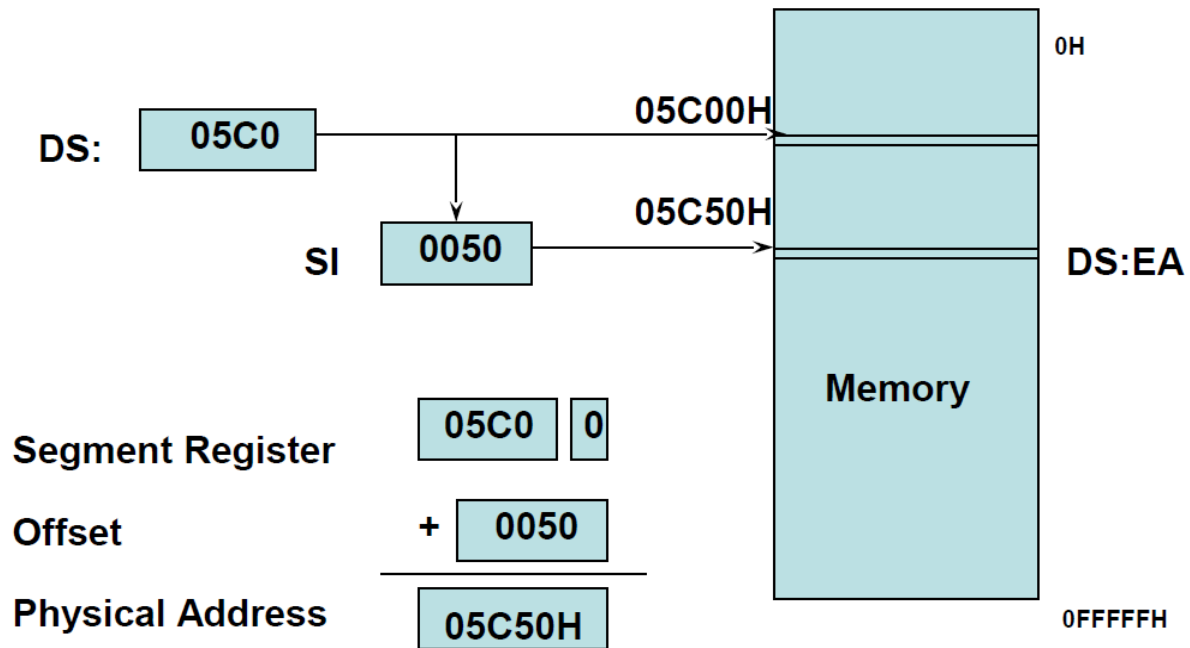
CS	3	4	8	A	0
IP +		4	2	1	4
Instruction (code) address	3	8	A	B	4
DS	1	2	3	4	0
DI +		0	0	2	2
Data address	1	2	3	6	2

SS	5	0	0	0	0
SP +		F	F	E	0
Stack address	5	F	F	E	0

Physical address generation for CS



Physical address generation for DS

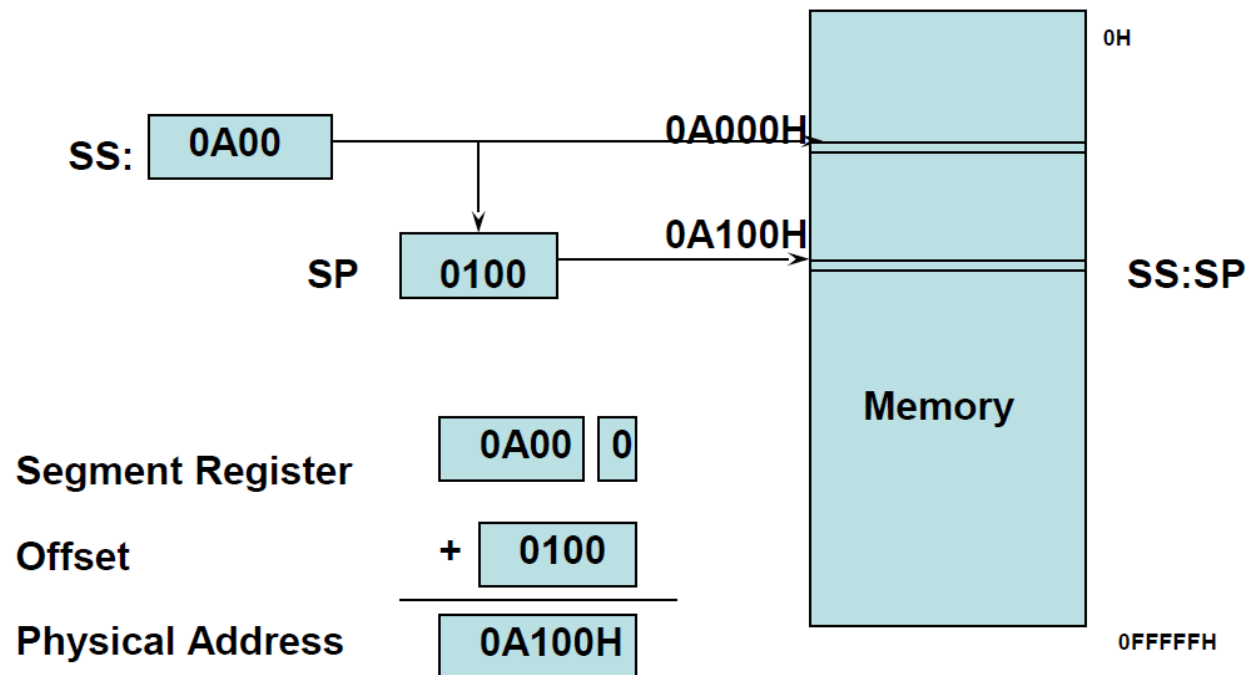


Data is fetched with respect to the DS register which contains starting or base address

The effective address (EA) or offset is in SI (default register for DS)

The EA depends on the addressing mode

Physical address generation for SS



The offset is given by the SP register.

The stack is always referenced with respect to the stack segment register.

The stack grows toward decreasing memory locations.

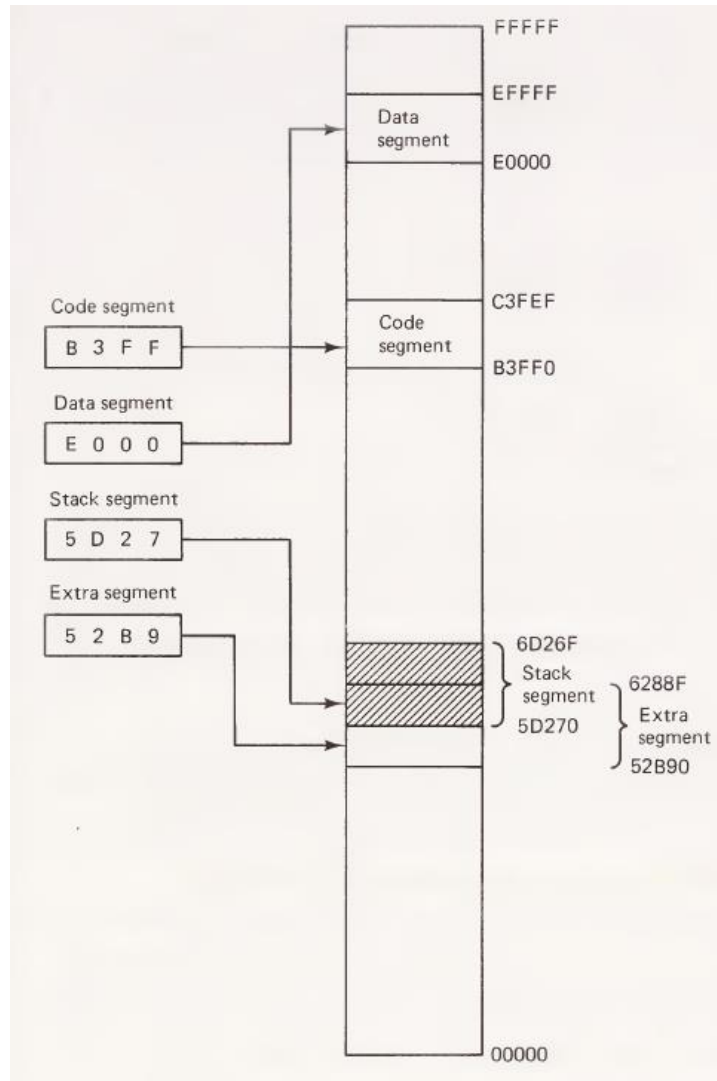
The SP points to the last or top item on the stack.

Calculation of Physical address-examples

Calculate the beginning and ending addresses for the data segment assuming that register DS=E000h

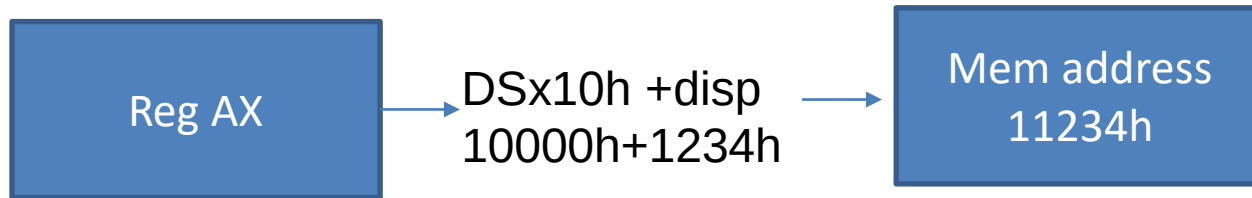
IF the content of CS register is B3FFh and the logical address is 05, calculate the physical address (real address)

Four segments of 8086 memory



Direct addressing

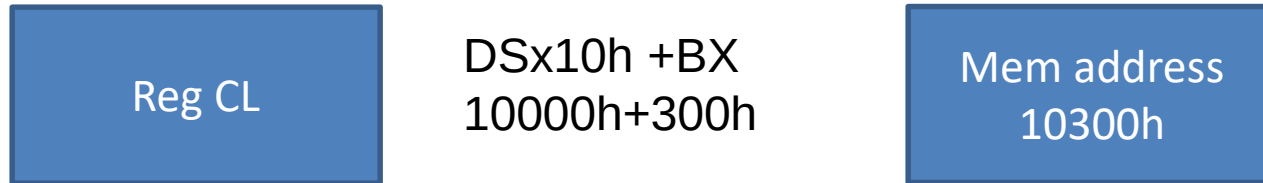
- Moves a byte or a word between a memory location located within the data segment and a register
- The instruction set does not support memory to memory transfer except for MOVS instruction (string instruction)
- Example : MOV[1234h], AX



- MOV AL, DATA loads AL from data segment memory location DATA[1234h]
- Memory location DATA is a symbolic memory location, while 1234h is the actual location

Register indirect addressing mode

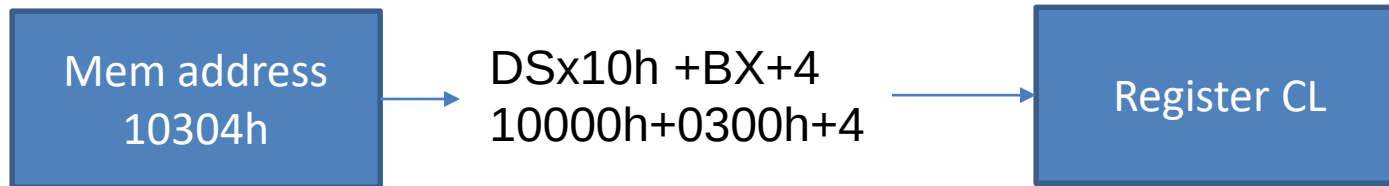
- Transfers a byte or a word between a register and a memory location addressed by an index register or base register
- The index and base registers are BP, BX, SI and DI
- Example : MOV [BX], CL



- MOV CX, [SI]
- MOV AX, [BP]
- MOV [SI], BX
- MOV AL,[BX]

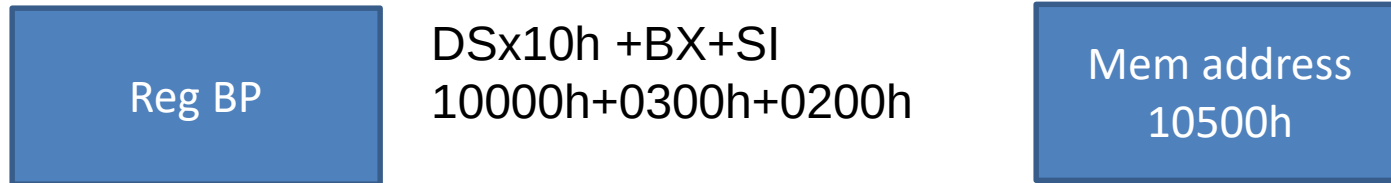
Register Relative addressing mode

- Moves a byte or word between a register and the memory location addressed by an index or base register plus displacement
- Ex: MOV AX, [BX+4] Bx=0300h
- MOV AX, ARRAY[BX]



Base Plus Index Addressing mode

- Transfers a byte or word between a register and the memory location addressed by a base register (BP or BX) plus an index register (SI or DI)
- Ex: MOV [BX+DI],CL
- MOV [BX+SI],BP



- The base register often holds the beginning location of a memory array, while the index register holds the relative position of an element in the array

Locating array data using base+index addressing

- A major use of the base+index addressing mode is to address elements in a memory array
- Suppose that the elements in an array located in the data segment at memory location **ARR** must be accessed
- To accomplish this, load BX register with the beginning address of the array and DI with the address of the data (element to be accessed) in the array.

Locating array data using base+index addressing

- MOV BX, Offset ARR
- MOV DI, 10h
- MOV AL, [BX+DI]
- MOV DI, 20h
- MOV [BX+DI],AL

Base Relative +Index addressing

- Transfers a byte/word between a register and a memory location addressed by a base and an index register plus displacement
- Ex: MOV AX, ARRAY[BX+DI]
- MOV AX, [BX+DI+4]

Reg DX

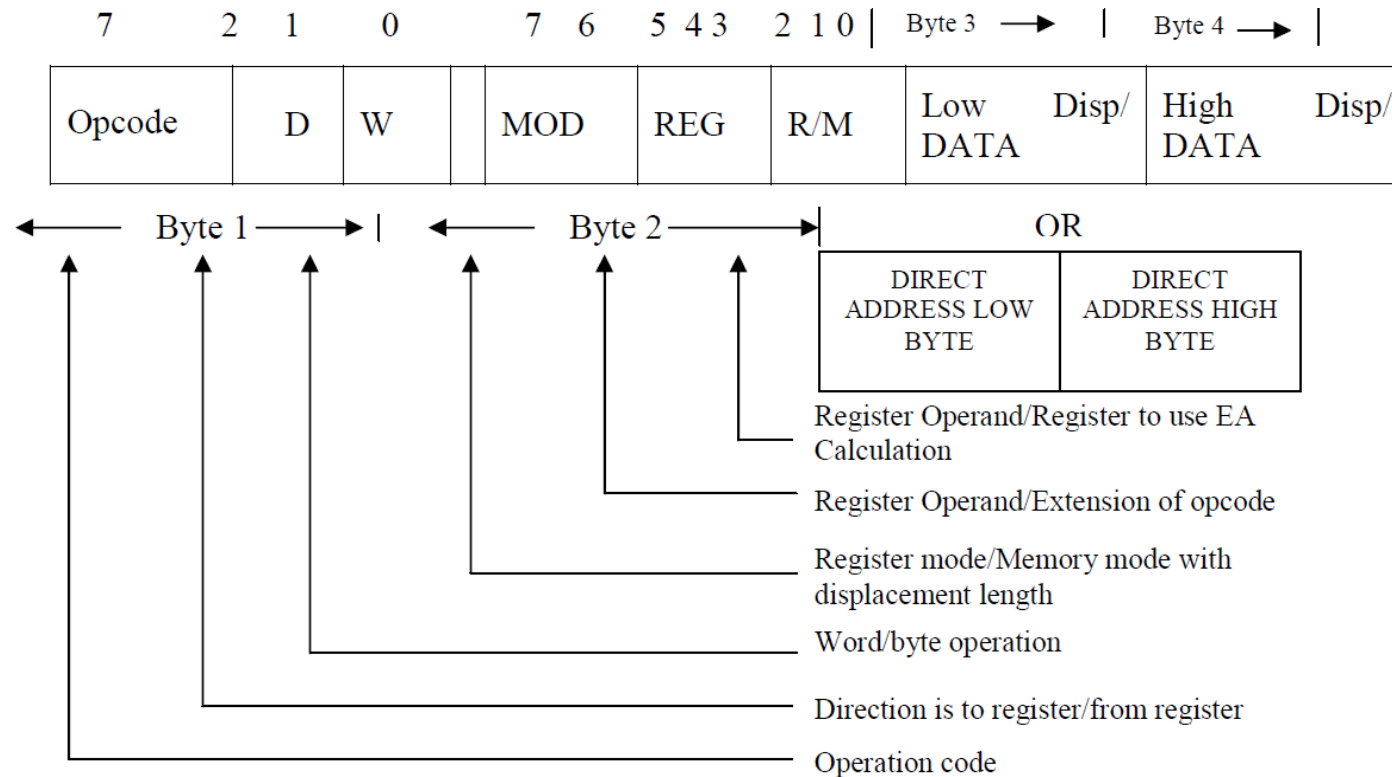
DSx10h +ARRAY+BX+SI
10000h+0300h+0200h

Mem address
10500h

- MOV [BX+SI+10h], AX

Instruction Format

- The 8086 instruction sizes vary from one to six bytes. Depending on the type of coding, an instruction may have more than one Hexcode



Instruction Format

- The OP code field occupies 6-bits. It defines the operation to be carried out by the instruction.
- Register Direct bit (D) occupies one bit. It defines whether the register operand in byte 2 is the source or destination operand.
- D=1 Specifies that the register operand is the destination operand.
- D=0 indicates that the register is a source operand.
- Data size bit (W) defines whether the operation to be performed is an 8 bit or 16 bit data
- W=0 indicates 8 bit operation
- W=1 indicates 16 bit operation

Instruction set

- An instruction is a binary pattern designed inside a microprocessor to perform a specific function.
- The entire group of instructions that a microprocessor supports is called Instruction Set.
- 8086 has more than 20,000 instructions

Classification of Instruction Set

- Data Transfer Instructions
- Arithmetic Instructions
- Bit Manipulation Instructions
- Program Execution Transfer Instructions
- String Instructions
- Processor Control Instructions

Data Transfer Instructions

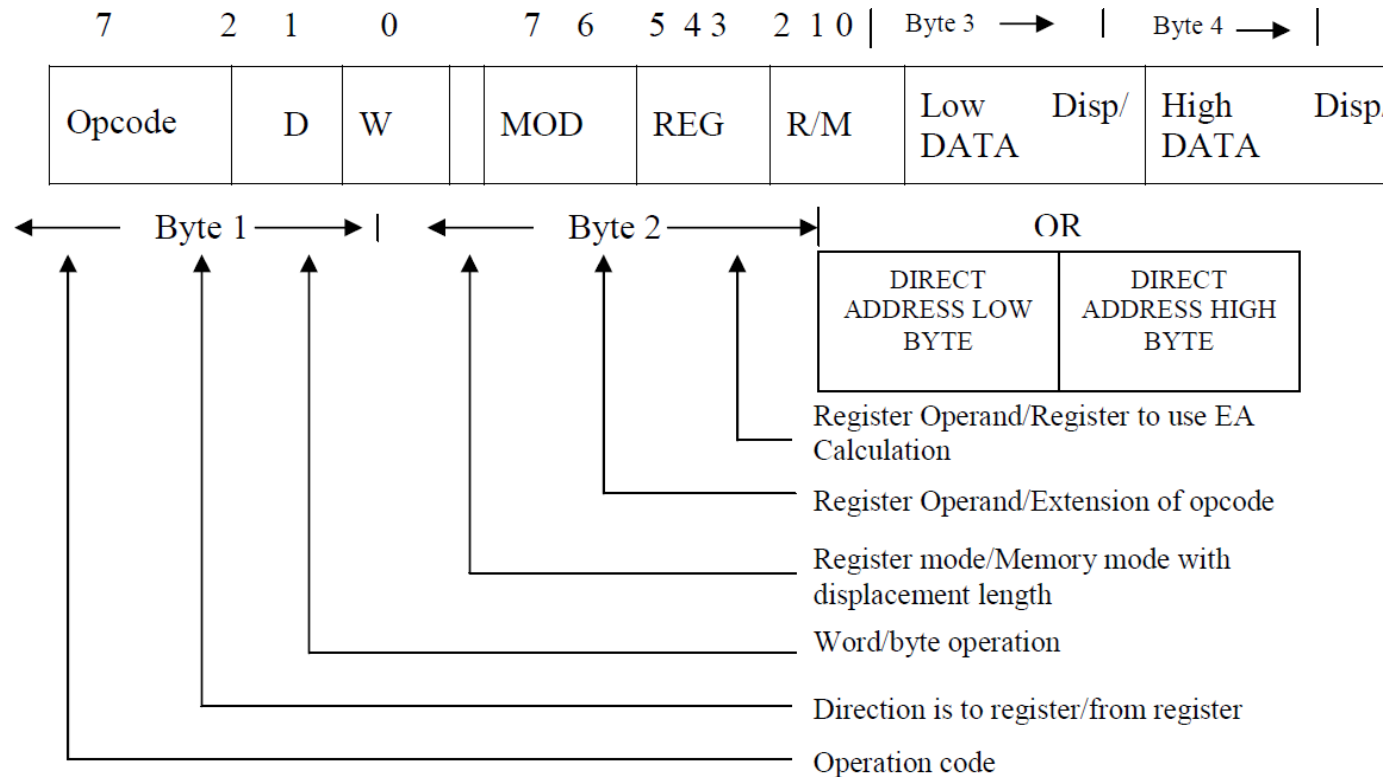
- These instructions are used to transfer data from source to destination.
- The operand can be a constant, memory location, register or I/O port address.

Data Transfer Instructions

- MOV Des, Src
- Src operand can be register, memory location or immediate operand.
- Des can be register or memory operand.
- Both Src and Des cannot be memory location at the same time.
- E.g.: MOV CX, 037A H
- MOV AL, BL
- MOV BX, [0301 H]

Instruction Format

- The 8086 instruction sizes vary from one to six bytes. Depending on the type of coding, an instruction may have more than one Hexcode



Instruction Format contd..



- The opcode selects the operation performed by the up
- It is 1 or 2 bytes long for most machine language instructions
- The general form of the first opcode byte is shown below



- First 6 bits of the 1st byte are the binary opcode.
- The remaining 2 bits indicate the direction (D) of the data flow and (W) indicates whether a data is a byte of word

Instruction Format contd..

- The OP code field occupies 6-bits. It defines the operation to be carried out by the instruction.
- Register Direct bit (D) occupies one bit. It defines whether the register operand in byte 2 is the source or destination operand.
- D=0 indicates that the register **specified by REG** is a source operand.
- D=1 Specifies that the register **specified by REG** is the destination operand.
- Data size bit (W) defines whether the operation to be performed is an 8 bit or 16 bit data
- W=0 indicates 8 bit operation



1st byte of the instruction code



2nd byte of the instruction code

MOD field

- specifies the addressing mode for the instruction

MOD	Function
00	No displacement
01	8 bit sign extended displacement
10	16 bit displacement
11	R/M is a register

Register assignments for the REG field and R/M field (MOD=11)

REG	W = 0	W = 1
000	AL	AX
001	CL	CX
010	DL	DX
011	BL	BX
100	AH	SP
101	CH	BP
110	DH	SL
111	BH	DI

R/M Memory addressing

R/M	MOD=00	MOD=01	MOD=10
000	(BX)+(SI)	(BX)+(SI)+D8	(BX)+(SI)+D16
001	(BX)+(DI)	(BX)+(DI)+D8	(BX)+(DI)+D16
010	(BP)+(SI)	(BP)+(SI)+D8	(BP)+(SI)+D16
011	(BP)+(DI)	(BP)+(DI)+D8	(BP)+(DI)+D16
100	(SI)	(SI)+D8	(SI)+D16
101	(DI)	(DI)+D8	(DI)+D16
110	DIRECT ADDRESS	(BP)+D8	(BP)+D16
111	(BX)	(BX)+D8	(BX)+D16

Arithmetic Instructions

- **ADD – ADD Destination, Source**
 - **ADC – ADC Destination, Source**
 - These instructions add a number from some *source* to a number in some *destination* and put the result in the specified destination.
 - The ADC also adds the status of the carry flag to the result.
 - The source may be an immediate number, a register, or a memory location.
 - The destination may be a register or a memory location.
 - The source and the destination in an instruction cannot both be memory locations.
 - The source and the destination must be of the same type (bytes or words).
 - Flags affected: AF, CF, OF, SF, ZF.
-
- ⑩ **ADD AL, 74H** Add immediate number 74H to content of AL. Result in AL
 - ⑩ **ADC CL, BL** Add content of BL plus carry status to content of CL
 - ⑩ **ADD DX, BX** Add content of BX to content of DX
 - ⑩ **ADD DX, [SI]** Add word from memory at offset [SI] in DS to content of DX
 - ⑩ **ADC AL, PRICES [BX]** Add byte from effective address PRICES [BX] plus carry status to content of AL
 - ⑩ **ADD AL, PRICES [BX]** Add content of memory at effective address PRICES [BX] to AL

SUB – SUB Destination, Source , SBB – SBB Destination, Source

- These instructions subtract the number in some *source* from the number in some *destination* and put the result in the destination.
- The SBB instruction also subtracts the content of carry flag from the destination.
- The source may be an immediate number, a register or memory location.
- The destination can also be a register or a memory location.
- However, the source and the destination cannot both be memory location.
- The source and the destination must both be of the same type (bytes or words).
- If you want to subtract a byte from a word, you must first move the byte to a word location such as a 16-bit register and fill the upper byte of the word with 0's. Flags affected: AF, CF, OF, PF, SF, ZF.
- SUB CX, BX CX – BX; Result in CX
- SBB CH, AL Subtract content of AL and content of CF from content of CH. Result in CH
- SUB AX, 3427H Subtract immediate number 3427H from AX
- SBB BX, [3427H] Subtract word at displacement 3427H in DS and content of CF from BX
- SUB PRICES [BX], 04H Subtract 04 from byte at effective address PRICES [BX],
- if PRICES is declared with DB; Subtract 04 from word at effective address PRICES [BX], if it is declared with DW.

- SBB CX, TABLE [BX] Subtract word from effective address TABLE [BX] and status of CF from CX.
- SBB TABLE [BX], CX Subtract CX and status of CF from word in memory at effective address TABLE[BX].